Training Production-Ready Large Language Models

A Comprehensive Guide to Modern Techniques (2025–2026)

From Data Curation to Deployment

Pre-training • Fine-tuning • Alignment • Inference Optimization

April 2026

Contents

Preface **ix**

I Foundations **1**

1 The LLM Training Landscape in 2026 **3**
1.1 The Evolution of Language Model Training . . . 3
1.2 The Modern Training Pipeline . . . 3
1.3 Scaling Laws and the Compute-Optimal Frontier . . . 4
1.4 Cost Realities . . . 4

2 Transformer Architecture – The Foundation **5**
2.1 The Decoder-Only Transformer . . . 5
2.1.1 Architectural Comparison: Key Modern Models . . . 6
2.2 Key Architectural Components . . . 6
2.2.1 Multi-Head Attention Variants . . . 6
2.2.2 Positional Encodings . . . 6
2.2.3 Normalization and Activation . . . 7
2.3 Mixture of Experts (MoE) . . . 7
2.4 Context Length and Efficient Attention . . . 8

II Data – The Foundation of Everything **9**

3 Data Curation – The Oil for the Engine **11**
3.1 Why Data Quality Trumps Quantity . . . 11
3.2 Data Sources . . . 11
3.3 Cleaning and Filtering Pipeline . . . 12
3.3.1 Levels 0–3: Rule-Based . . . 12
3.3.2 Levels 4–5: Model-Based . . . 12
3.3.3 Levels 6+: LLM-Driven (2025–2026 Frontier) . . . 12
3.4 Deduplication . . . 12
3.5 Multi-Stage Training . . . 12

4 Tokenization – From Text to Vectors **15**

4.1 Subword Tokenization . . . 15
4.1.1 Vocabulary Size Trade-offs . . . 15
4.2 Universal Tokenizers . . . 16
4.3 Sequence Packing . . . 16

5 Synthetic Data Generation 17
5.1 The Case for Synthetic Data . . . 17
5.2 Core Techniques . . . 18
5.2.1 Self-Instruct and Alpaca . . . 18
5.2.2 Evol-Instruct (WizardLM) . . . 18
5.2.3 Magpie: Model Self-Generation . . . 18
5.2.4 Orca-Style Chain-of-Thought Augmentation . . . 18
5.2.5 Synthetic Math and Code Chains . . . 18
5.2.6 Persona-Driven Diversity . . . 19
5.3 Quality Filtering . . . 19
5.4 Risks: Model Collapse and Bias Amplification . . . 19
5.5 Synthetic Data Composition Recommendations . . . 20

III Pre-training 21

6 Pre-training Objectives and Strategies 23
6.1 Next-Token Prediction . . . 23
6.2 Reinforcement Pre-Training (RPT) . . . 24
6.3 Curriculum Learning . . . 24
6.4 Instruction-Response Augmented Pre-training . . . 24
6.5 Long-Context Pre-training . . . 24

7 Distributed Training and Infrastructure 25
7.1 Parallelism Strategies . . . 25
7.1.1 3D Parallelism . . . 26
7.2 Mixed-Precision Training . . . 26
7.3 Checkpointing and Fault Tolerance . . . 26
7.4 Communication Optimization . . . 26
7.5 Optimizers and Learning Rate Schedules . . . 26
7.5.1 Optimizer Selection . . . 26
7.5.2 Learning Rate Schedule . . . 27
7.5.3 Gradient Clipping and Stability . . . 27
7.6 Key Training Frameworks . . . 28

IV Post-Training – From Base Model to Product 29

8 Supervised Fine-Tuning (SFT) 31
8.1 Purpose and Principles . . . 31
8.2 Data Quality for SFT . . . 31
8.3 Chat Templates . . . 32

8.4 Parameter-Efficient Fine-Tuning (PEFT) 32

9 Alignment – RLHF, DPO, and Beyond **33**
9.1 The Alignment Problem 33
9.2 Reward Modeling 33
9.3 RLHF 34
9.4 DPO 34
9.5 GRPO 34
9.6 DAPO 35
9.7 RLAIF and Constitutional AI 35
9.8 RLVR 35
9.9 The Modern Alignment Stack 35

10 Training for Reasoning **37**
10.1 The Rise of Reasoning Models 37
10.2 Chain-of-Thought (CoT) 38
10.3 Process Reward Models (PRM) 38
10.4 Test-Time Search 38
10.5 TLT – Accelerating Training 38
10.6 R1-Style Pure RL Training 39
10.7 Thinking Tokens (Quiet-STaR) 39

11 Knowledge Distillation and Model Compression **41**
11.1 Why Distillation Matters 41
11.2 Response (Output-Level) Distillation 42
11.3 Logit-Level Distillation 42
11.4 Sequence-Level Knowledge Distillation (SeqKD) 42
11.5 MiniLLM: Reverse KL for Distillation 42
11.6 Intermediate Layer Distillation 43
11.7 On-Policy Distillation 43
11.8 Speculative Decoding as Distillation 43
11.9 Practical Distillation Workflow 43

V Evaluation, Safety & Deployment **45**

12 Evaluating LLMs – Benchmarks and Beyond **47**
12.1 LLM-as-Judge 48
12.2 Chatbot Arena and ELO Rankings 48
12.3 Benchmark Contamination and Gaming 48

13 Safety, Ethics, and Constitutional AI **49**
13.1 Safety Taxonomy 49
13.2 Constitutional AI (CAI) 49
13.3 Red-Teaming 50
13.4 Interpretability and Explainability 50
13.5 Safe Deployment Practices 50

14 Inference and Deployment at Scale **51**
14.1 Quantization . . . 52
14.2 Inference Frameworks . . . 52
14.3 Speculative Decoding . . . 52
14.4 Continuous Batching . . . 52
14.5 KV Cache Management . . . 52
14.6 Multi-GPU Inference . . . 53

VI Advanced Topics and Future Directions **55**

15 Specialized Domains and Multi-modality **57**
15.1 Domain-Specific Pre-training . . . 57
15.2 Multimodal Models . . . 58
15.3 Agentic AI . . . 58

16 Model Merging and Recombination **59**
16.1 Merging Methods . . . 59
16.2 Task Vectors . . . 60
16.3 Practical Merging Workflow . . . 60

17 Continual Learning and Lifelong Adjustment **61**
17.1 The Forgetting Problem . . . 61
17.2 Mitigation Strategies . . . 61
17.3 Continual Pre-training . . . 62

18 The Future of LLM Training **63**
18.1 Dominant Trends . . . 63
18.2 Open Challenges . . . 64
18.3 The Next Frontier: 2026–2028 . . . 64

19 Case Studies and Frontier Lab Analysis **65**
19.1 OpenAI – GPT-5 Family . . . 65
19.2 DeepSeek – R1 and V3 . . . 66
19.3 Meta – Llama 4 Family . . . 66
19.4 Anthropic – Claude Family . . . 66
19.5 Google DeepMind – Gemini Family . . . 67
19.6 Alibaba / Qwen – Qwen 3 Family . . . 67
19.7 Cross-Cutting Patterns . . . 67

A Glossary of Key Terms **69**

B Recommended Reading **71**

C Hardware Reference **73**

D Quick-Start Recipes **75**

D.1 QLoRA SFT with TRL and PEFT . . . 75
D.2 DPO Alignment with TRL . . . 76
D.3 GRPO Reasoning Training with TRL . . . 76
D.4 Serving with vLLM . . . 77
D.5 Model Merging with mergekit . . . 77
D.6 Continual Pre-training on Domain Data . . . 78

E Cloud Provider Reference **81**
E.1 Overview . . . 81
E.2 Reservation vs. Spot Strategies . . . 81
E.3 Cost Estimation: Representative Workloads . . . 82
E.4 Infrastructure Checklist . . . 82

F Training Hyperparameter Reference **83**
F.1 Pre-training . . . 83
F.2 Supervised Fine-Tuning (SFT) . . . 84
F.3 LoRA / QLoRA . . . 84
F.4 DPO / SimPO Alignment . . . 85
F.5 GRPO Reasoning Training . . . 85
F.6 Common Failure Modes and Fixes . . . 86

G The Implementation Treasury: Theory & Code **87**
G.1 Shared Foundation . . . 87
G.2 Optimization Objectives . . . 87
G.2.1 Stage 1 — Pre-training . . . 87
G.2.2 Perplexity (PPL) . . . 88
G.2.3 Stage 3 — Preference Optimisation . . . 88
G.3 Attention Mechanisms . . . 88
G.3.1 Scaled Dot-Product Attention . . . 88
G.3.2 Multi-Head Attention (MHA) . . . 89
G.3.3 Grouped Query Attention (GQA) . . . 90
G.3.4 Multi-Query Attention (MQA) . . . 90
G.3.5 Multi-head Latent Attention (MLA) . . . 91
G.4 Positional Encodings . . . 91
G.4.1 Rotary Position Embedding (RoPE) . . . 91
G.4.2 ALiBi (Attention with Linear Biases) . . . 92
G.5 Normalization and Activation . . . 92
G.5.1 RMSNorm . . . 92
G.5.2 SwiGLU Feed-Forward Network . . . 93
G.6 Optimization . . . 93
G.6.1 AdamW . . . 93
G.6.2 Muon Optimizer . . . 93
G.6.3 Lion Optimizer . . . 94
G.6.4 Cosine Schedule with Warmup . . . 94
G.7 Parameter-Efficient Fine-Tuning . . . 94
G.7.1 LoRA (Low-Rank Adaptation) . . . 94

G.7.2 DoRA (Weight-Decomposed Low-Rank Adaptation) 95
G.7.3 PPO (Proximal Policy Optimization) 95
G.8 Alignment Objectives 95
G.8.1 DPO (Direct Preference Optimization) 95
G.8.2 SimPO (Simple Preference Optimization) 95
G.8.3 GRPO (Group Relative Policy Optimisation) 96
G.8.4 KTO (Kahneman-Tversky Optimization) 97
G.9 Inference and Distillation 97
G.9.1 Knowledge Distillation (Forward KL) 97
G.9.2 MiniLLM (Reverse KL) 97
G.9.3 KV Cache Management 97
G.9.4 Quantization Scaling 97
G.9.5 Speculative Decoding 98
G.10 Model Merging 98
G.10.1 Task Vectors 98
G.10.2 SLERP (Spherical Linear Interpolation) 98
G.10.3 DARE (Drop And REscale) 99
G.10.4 TIES-Merging 99
G.11 Continual Learning — EWC 99
G.11.1 Elastic Weight Consolidation (EWC) 99
G.12 Final Assembly — TinyGPT 100

References **101**

List of Tables

1.1 Typical cost ranges for LLM training components 4

2.1 Representative modern model architectures 6
2.2 Comparative analysis of activation functions in LLMs 7
2.3 Approximate KV cache memory at different context lengths (BF16, 32 heads) 8

3.1 Typical data source composition for pre-training 12

5.1 Recommended SFT data mix composition using synthetic sources 20

7.1 Optimizer memory footprint and speed comparison (relative to model size) . 28
7.2 Key training frameworks in 2025–2026 28

9.1 The modular alignment stack in 2026 35

12.1 Standard LLM evaluation benchmarks 48

14.1 Quantization formats and trade-offs 52

19.1 Comparison of frontier lab training approaches 68

C.1 GPU and accelerator reference for LLM training (TFLOPS are peak BF16/FP16) 73

E.1 Cloud GPU pricing reference (April 2026; prices indicative, verify current rates) 81
E.2 Cost estimates for representative LLM workloads (indicative, 2026) 82

F.1 Pre-training hyperparameter reference 83
F.2 SFT hyperparameter reference 84
F.3 LoRA and QLoRA hyperparameter reference 84
F.4 DPO and SimPO hyperparameter reference 85
F.5 GRPO hyperparameter reference 85
F.6 Common training failure modes and remediation 86

Preface

The large language model revolution has moved faster than any previous technology cycle in machine learning. What required a year-long research effort at a top lab in 2022 now fits in a weekend hackathon; models that once cost $10 million to train can be replicated for tens of thousands of dollars.

This guide is written for engineers and researchers who need to *do the work*—not survey it from a distance. It covers the full training stack: data pipelines, pre-training at scale, the post-training alignment stack, evaluation, safety, and production deployment. Each chapter synthesizes the most important findings from 2024–2026 alongside the enduring principles from earlier years.

Who this book is for: ML engineers building fine-tuning pipelines; researchers designing new alignment methods; infrastructure engineers provisioning GPU clusters; technical leads making architectural decisions for LLM-powered products.

How to read this book: This guide is designed for both linear reading and targeted reference.

- **Parts I–III (Foundations & Pre-training):** Essential for teams building or modifying base models. Covers data, architecture, and scaling.
- **Parts IV–V (Post-training & Alignment):** The high-value stack for 2026. Relevant to everyone from hobbyists to frontier researchers.
- **Part VI (Advanced Topics):** Specialized research on reasoning, multi-modality, and the 2027 roadmap.

Each chapter begins with a **What You Will Learn** overview and a **Mini Table of Contents** for quick navigation. Mathematical formulations are concentrated in Appendix G.2–G.11, with direct links provided throughout the text. Ready-to-use PyTorch code exists in Appendix H, serving as the executable companion to the theoretical chapters.

Part I

Foundations

Chapter 1

The LLM Training Landscape in 2026

What You Will Learn

- Understand the 2026 shift from parameter-count to inference-optimality.
- Review the 5-stage modern training pipeline (Pre-training to Deployment).
- Analyze Chinchilla scaling laws and their 2026 industrial overrides.
- Estimate the compute and human capital costs of frontier LLM development.

1.1 The Evolution of Language Model Training

Large language models have undergone a dramatic transformation. What began as research curiosities have become the computational backbone of search engines, coding assistants, data analysis platforms, and creative tools. The global market for LLMs was valued at $6.4 billion in 2024 and is projected to reach $36.1 billion by 2030.

The training paradigm has shifted substantially. Early models relied on a simple recipe: more data plus more parameters equals better performance. By late 2025, several hard constraints became apparent. Marginal gains from more compute diminish. High-quality, deduplicated text is finite and increasingly expensive. And ever-larger models collide with real-time product requirements and energy budgets.

Post-training—the combination of supervised fine-tuning, preference optimization, and reinforcement learning—now accounts for the majority of a model's usable capability. The field has moved from "make the base model bigger" to "given a strong base model, how do we make it safe, efficient, and excellent at specific jobs?"

1.2 The Modern Training Pipeline

1. **Data Curation & Pre-training:** Next-token prediction on trillions of tokens.

2. **Supervised Fine-Tuning (SFT):** Instruction-response pairs for baseline behavior.
3. **Preference Optimization:** DPO, SimPO, or KTO for human value alignment.
4. **Reinforcement Learning:** Verifiable rewards or environment feedback for reasoning.
5. **Evaluation, Safety & Deployment:** Benchmarking, red-teaming, quantization, serving.

1.3 Scaling Laws and the Compute-Optimal Frontier

The Chinchilla scaling laws (Hoffmann et al. 2022) established that for a fixed compute budget C, optimal model size N and training tokens D scale as $N \propto C^{0.5}$ and $D \propto C^{0.5}$—meaning earlier models were systematically undertrained. Chinchilla-70B (70B parameters, 1.4T tokens) (Hoffmann et al. 2022) outperformed GPT-3-175B (175B parameters, 300B tokens) (Brown et al. 2020) at one-quarter the inference cost.

By 2025–2026, frontier labs diverge from strict Chinchilla prescriptions for a practical reason: *inference cost dominates training cost at scale.* Training a smaller model on more tokens is cheaper to deploy even if it requires somewhat more training compute.

Scaling Law Implications for Practitioners

- Smaller models trained longer ("overtrained" relative to Chinchilla) are cheaper to deploy.
- At 7B parameters, the Chinchilla-optimal token count is ∼140B; in practice, training on 1T+ tokens delivers significantly better downstream performance.
- Emergent capabilities appear discontinuously at scale, making capability predictions unreliable.
- Data quality exerts a multiplicative effect: doubling data quality is often more impactful than doubling data quantity.

1.4 Cost Realities

Component	Cost Range	Key Driver
Pre-training (frontier)	\$5M – \$100M+	Compute (GPU hours)
Pre-training (7B–70B)	\$50K – \$5M	Data quality + GPU hours
Fine-tuning (LoRA/QLoRA)	\$100 – \$10K	Dataset size, GPU type
RLHF / DPO alignment	\$10K – \$500K	Human annotation costs
Evaluation & Red-teaming	\$5K – \$100K	Evaluator complexity
Inference infrastructure	\$1K – \$50K/mo	Traffic, latency SLA

Table 1.1: Typical cost ranges for LLM training components

Chapter 2

Transformer Architecture – The Foundation

Contents

1.1 The Evolution of Language Model Training 3

1.2 The Modern Training Pipeline 3

1.3 Scaling Laws and the Compute-Optimal Frontier 4

1.4 Cost Realities 4

What You Will Learn

- Evaluate the decoder-only transformer and its 2026 architectural variations.
- Analyze MQA, GQA, and MLA for efficient KV cache management at scale.
- Compare modern positional encodings including RoPE, iRoPE, and ALiBi.
- Review production-standard normalization (RMSNorm) and activation (SwiGLU) functions.
- Understand the 2025 transition to fine-grained Mixture of Experts (MoE).

2.1 The Decoder-Only Transformer

Nearly all modern autoregressive LLMs use a decoder-only transformer architecture: sequential blocks of masked self-attention and feed-forward sub-layers. In PaLM-540B, approximately 90% of parameters reside in feed-forward layers—which is why MoE focuses on FFN efficiency.

2.1.1 Architectural Comparison: Key Modern Models

Model	Params	Context	Notable Features
Llama 3.1 (70B)	70B	128K	GQA, RoPE, SwiGLU
Mistral 7B	7B	32K	GQA, sliding window
Mixtral 8x7B	47B total	32K	MoE, 13B active
DeepSeek-V3	671B total	128K	Fine-grained MoE, MLA
Llama 4 Maverick	400B total	1M	MoE, iRoPE, 128 experts

Table 2.1: Representative modern model architectures

2.2 Key Architectural Components

2.2.1 Multi-Head Attention Variants

Standard multi-head attention (MHA) uses H heads each with independent Q, K, V projections (formulation: Appendix G.3; code: Appendix H). The KV cache grows as $2 \times H \times d_{\text{head}} \times L$ per token at inference—a critical bottleneck at long context.

- **Multi-Query Attention (MQA):** Single K,V head shared by all query heads. Maximum KV cache reduction; can hurt quality at small scale (formulation §G.3.4).
- **Grouped Query Attention (GQA):** G KV head groups ($1 < G < H$) (Ainslie et al. 2023). Balances cache reduction and quality. Used in Llama 3, Mistral, and most 2025 models (formulation §G.3.3, code Listing G.5).
- **Multi-head Latent Attention (MLA):** DeepSeek-V3. KV cache is compressed into a low-rank latent vector c_{KV}, from which heads are projected via an up-projection matrix W_{UK}. This achieves the memory footprint of MQA while maintaining the expressive power of MHA (see formulation §G.3.5, code Listing G.7).

MLA Mechanics

MLA's core innovation is the decoupled latent vector:

$$c_{KV} = W_{DK} h_t, \quad k_t, v_t = W_{UK} c_{KV}$$

where W_{DK} is a down-projection and W_{UK} an up-projection. RoPE is applied to a separate, non-compressed portion of the query and key to maintain relative position awareness without bloating the KV cache.

2.2.2 Positional Encodings

Full derivations in Appendix G.4.

- **RoPE (Rotary Position Embedding):** Applies rotation matrices to Q/K vectors (Su et al. 2024). Encodes relative positions implicitly. Dominant 2025–2026 standard.

- **iRoPE (Llama 4):** Interleaved no-position and RoPE layers. Enables 10M-token context without explicit positional interpolation.
- **ALiBi:** Linear bias on attention scores (Press et al. 2022). Zero extra parameters, strong length generalization.
- **YaRN / LongRoPE:** NTK-aware RoPE interpolation for 4–8× context extension without full retraining (Peng et al. 2023).

2.2.3 Normalization and Activation

Full formulations in Appendix G.5 (RMSNorm, SwiGLU).

- **RMSNorm with pre-normalization:** More stable than post-norm LayerNorm. Dominant in all major 2025 models (formulation §G.5.1).
- **SwiGLU:** Gated FFN activation combining Swish and GLU (Shazeer 2020). Standard for 2024–2026 (formulation §G.5.2).

Function	Formulation	Key Trade-off / Usage
ReLU	$\max(0, x)$	Simple, fast. Risk of "dying ReLU" (zero gradient).
GeLU	$x\Phi(x)$	Smooth, probabilistic gating. GPT-2/3 standard.
Swish / SiLU	$x\sigma(x)$	Non-monotonic, smoother than ReLU. Llama 1/2.
SwiGLU	$\text{Gated}(x, \text{Swish})$	Most expressive. 2026 production standard.

Table 2.2: Comparative analysis of activation functions in LLMs

2.3 Mixture of Experts (MoE)

Replace the dense FFN with multiple smaller expert networks and a router selecting which experts process each token (Shazeer et al. 2017). Only 1–2 of 8–64 experts activate per token. Mixtral 8x7B: 47B total parameters, ∼13B active per forward pass.

Key MoE Developments in 2025–2026

- **DeepSeekMoE:** Fine-grained experts with shared expert isolation. 256 routed experts + 1 shared expert per layer.
- **DeepSeekMoE (Dai et al. 2024):** Fine-grained experts with shared expert isolation. 256 routed experts + 1 shared expert per layer.
- **Sparse Upcycling:** Convert dense models to MoE without training from scratch.
- **SwitchHead:** MoE applied to attention projection layers (Q, K, V).
- **Expert Parallelism:** Distributed experts across GPUs for efficient scale.

- **Load Balancing:** Auxiliary loss or expert-choice routing prevents expert collapse.

2.4 Context Length and Efficient Attention

- **Flash Attention (v2/v3):** $O(n^2) \to O(n)$ memory via IO-aware tiling (Dao 2024; Dao et al. 2022; Shah et al. 2024). Mandatory for modern training. FA3 adds warp-specialization for $\sim$2$\times$ speedup over FA2 on H100.
- **Ring Attention:** Million-token contexts across devices in a ring topology. Each device holds one segment; KV chunks circulate.
- **Sliding Window Attention:** Fixed local window with cross-layer information flow (Mistral).

Context Window	KV Cache (7B)	Use Case
4K tokens	512 MB	Short conversations
32K tokens	4 GB	Document processing
128K tokens	16 GB	Full codebase analysis
1M tokens	128 GB	Entire book corpus

Table 2.3: Approximate KV cache memory at different context lengths (BF16, 32 heads)

Part II

Data – The Foundation of Everything

Chapter 3

Data Curation – The Oil for the Engine

Contents

2.1 The Decoder-Only Transformer 5
2.2 Key Architectural Components 6
2.3 Mixture of Experts (MoE) 7
2.4 Context Length and Efficient Attention 8

What You Will Learn

- Implement quality filtering and deduplication at the trillion-token scale.
- Review domain sampling proportions (Web, Code, Science, Books) for 2026.
- Analyze strategies for removing PII and toxic content while preserving knowledge.
- Understand the 2025 shift from quantity-driven to quality-driven curation.

3.1 Why Data Quality Trumps Quantity

FineWeb (Penedo et al. 2024) (15T tokens, heavily curated) outperforms RedPajama (20T tokens, lightly filtered)—5 trillion extra noisy tokens *hurt* performance. Modern frontier models train on 10–20T tokens, approaching available high-quality text limits.

3.2 Data Sources

Web crawl (FineWeb, DCLM), books and academic literature, code repositories (The Stack), curated knowledge bases (Wikipedia), and multilingual sources (FineWeb-2).

Source	Typical Mix	Notes
Web crawl	50–70%	Requires aggressive cleaning
Code	15–25%	Boosts reasoning and instruction following
Books	5–15%	Long-form coherence
Scientific papers	3–8%	Technical reasoning
Wikipedia / Wikidata	2–5%	Factual grounding
Curated instruction data	1–3%	Bridges to SFT

Table 3.1: Typical data source composition for pre-training

3.3 Cleaning and Filtering Pipeline

3.3.1 Levels 0–3: Rule-Based

URL filtering, language identification, boilerplate removal, heuristic quality filters (perplexity thresholds, line length statistics, repetition detection).

3.3.2 Levels 4–5: Model-Based

FastText/transformer classifiers for quality scoring. GneissWeb: sharded deduplication at 10T scale. SoftDedup: reweight rather than delete—preserves rare but informative examples.

3.3.3 Levels 6+: LLM-Driven (2025–2026 Frontier)

DataEvolve / Data Darwinism: autonomous LLM-driven quality analysis and iterative cleaning. Category-specific strategies (SwallowCode, MegaMath). Instruction-response augmented pre-training.

3.4 Deduplication

Exact (hash-based), near-duplicate (MinHash/LSH with GPU-accelerated connected components), and fuzzy (embedding-based similarity). Near-deduplication at the paragraph level catches reformatted reposts that exact dedup misses.

3.5 Multi-Stage Training

Stage 1: large corpus (10T+) for breadth. Stage 2: curated subset (0.5–1T) for depth and polish. Some labs add a Stage 3: task-specific mix (code-heavy, math-heavy) aligned to deployment use case.

Data Contamination

Evaluation benchmarks present in pre-training data inflate benchmark scores without improving actual capability. Decontaminate training data by removing near-exact matches to test sets. Always report decontamination procedures in training reports.

Chapter 4

Tokenization – From Text to Vectors

Contents

3.1 Why Data Quality Trumps Quantity 11
3.2 Data Sources 11
3.3 Cleaning and Filtering Pipeline 12
3.4 Deduplication 12
3.5 Multi-Stage Training 12

What You Will Learn

- Compare BPE, Unigram, and WordPiece tokenization algorithms.
- Master the 2026 standard 128K vocabularies for multilingual performance.
- Implement multi-byte character handling and special token management.
- Analyze the trade-offs between vocabulary size and inference latency.

4.1 Subword Tokenization

BPE (Sennrich et al. 2016) (dominant, GPT-series, 32K–128K vocabulary), WordPiece (BERT), SentencePiece (Kudo and Richardson 2018) (language-agnostic, multilingual models).

4.1.1 Vocabulary Size Trade-offs

Larger vocabularies reduce sequence length (faster attention) but increase embedding parameter count and can impair performance on rare tokens. 50K–100K is the 2025 sweet spot; some multilingual models use 200K+.

4.2 Universal Tokenizers

Up to 20.2% higher win rates in language adaptation. 5% improvement on unseen languages. Enables efficient post-training language expansion. Key design criterion: fertility ratio (tokens per word) should be ≤ 2 for all target languages.

4.3 Sequence Packing

Length-aware combinatorial optimization: 40% fewer truncations, improved modeling performance. Cross-document attention masking prevents spurious inter-document dependencies within packed batches.

Chapter 5

Synthetic Data Generation

Contents

4.1 Subword Tokenization 15
4.2 Universal Tokenizers 16
4.3 Sequence Packing 16

What You Will Learn

- Review the self-instruct and prompt-based data generation evolution.
- Master Magpie and PersonaHub for generating high-diversity instruction sets.
- Implement quality control and model collapse prevention strategies.
- Analyze the cost-efficiency of synthetic data vs. human annotation.

Frontier models are trained on trillions of tokens—but high-quality human-written text is finite. Synthetic data, generated or augmented by existing LLMs, now supplies a substantial fraction of both pre-training and post-training corpora.

5.1 The Case for Synthetic Data

- **Scale without annotation cost:** A single human expert can label ∼1K examples per day; a model can generate millions in minutes.
- **Long-tail coverage:** Rare instructions, specialized domains, and unusual reasoning patterns are under-represented in web crawls.
- **Consistency:** Synthetic data can be templated to enforce format, difficulty, and safety constraints uniformly.
- **Reasoning chain density:** Writing out step-by-step solutions is prohibitively slow for humans; LLMs produce them at scale.

5.2 Core Techniques

5.2.1 Self-Instruct and Alpaca

Self-Instruct (Wang et al. 2022) bootstraps instruction data from a seed set of 175 human-written examples, using an LLM to generate new (instruction, input, output) triples, then filtering by ROUGE-based deduplication. Stanford Alpaca (Taori et al. 2023) applied this to LLaMA-7B using GPT-3.5, generating 52K instruction-following examples for under $500. Despite the low cost, Alpaca matched InstructGPT-175B on several instruction-following metrics, demonstrating the asymmetric leverage of synthetic data.

5.2.2 Evol-Instruct (WizardLM)

Rather than generating new instructions from scratch, Evol-Instruct (Xu et al. 2023) mutates an existing instruction through four "evolution" operators: add constraints, deepen complexity, concretize, increase reasoning steps. Each mutation produces a harder variant. Training on evolved instructions produces models that significantly outperform models trained on the original dataset because they see a graduated difficulty curriculum automatically.

5.2.3 Magpie: Model Self-Generation

Magpie (Xu et al. 2024) exploits the observation that instruction-tuned models exposed to only a system prompt and role prefix will auto-complete realistic user instructions. The pipeline:

1. Feed the model only the system header + user-role prefix (e.g., `<|start_header_id|>user<|end_header_id|>`)
2. Sample a user instruction from the model's own distribution.
3. Feed the (system + generated instruction) back, sample the assistant response.
4. Filter by quality score (self-consistency, LLM-judge).

Magpie produces on-distribution synthetic data without any seed examples and scales to millions of examples cheaply. Models fine-tuned on Magpie data at the 8B scale have ranked competitively on AlpacaEval 2.

5.2.4 Orca-Style Chain-of-Thought Augmentation

Orca (Mukherjee et al. 2023) augments a dataset of short instruction-response pairs from FLAN by adding *detailed reasoning traces* generated by GPT-4. The key insight: small models benefit more from observing the reasoning process than from observing the final answer alone. Orca-2 and later OpenHermes models confirm that 200K–1M GPT-4-augmented examples with reasoning traces can match much larger instruction-tuned models.

5.2.5 Synthetic Math and Code Chains

For verifiable domains, synthetic chains can be validated automatically:

- **Math (MATH, GSM8K-style):** Generate problems via template + parameter randomization. Solve with a CAS (SymPy, Wolfram) to verify. Filter chains where the LLM-generated solution matches the verified answer.
- **Code:** Generate function docstrings and unit tests first, then generate implementations. Execute tests to verify correctness. Only passing solutions enter training data.

DeepSeek's reasoning data pipeline uses this approach: generate a candidate, verify via code execution or symbolic math, keep correct examples with their reasoning chains.

5.2.6 Persona-Driven Diversity

Persona Hub (Wang et al. 2024): maintain a library of 1 billion synthesized user personas (age, profession, goals, style). Sample personas at generation time to produce diverse instruction styles. Prevents the "assistant voice" homogenization that afflicts single-persona synthesis.

5.3 Quality Filtering

Raw synthetic data contains errors, repetition, and unsafe content. A multi-stage filter is essential:

1. **Format checks:** Regex validation for expected chat template structure.
2. **Length filtering:** Remove excessively short responses (<20 tokens) and runaway generations (>4K tokens without meaningful content).
3. **Self-consistency scoring:** Generate the response twice with different seeds; low cosine similarity flags inconsistent or hallucinated content.
4. **LLM-as-judge:** Score on helpfulness, accuracy, and safety (1–5 scale). Discard scores ≤ 3.
5. **Deduplication:** MinHash near-deduplication to prevent overfitting to repeated patterns.
6. **Human spot-check:** 1–2% random sample reviewed by domain experts to calibrate automated filters.

5.4 Risks: Model Collapse and Bias Amplification

Model Collapse

Training repeatedly on model-generated data without human-curated anchors leads to *model collapse* (Shumailov et al. 2024): the distribution narrows, variance decreases, and tail behaviors (rare but important knowledge) are lost. The antidote: always maintain a human-curated anchor set—typically 10–30% of the training mix—and monitor perplexity on a fixed held-out real-text corpus. If perplexity rises on the real-text held-out set, collapse is beginning.

Bias amplification: errors and biases in the teacher model are reproduced and reinforced in the student. Use diverse teacher models (not just one API) and human review to audit for systematic failures.

5.5 Synthetic Data Composition Recommendations

Synthetic Source	Mix %	Purpose
Magpie (self-generated)	20–40%	On-distribution instruction diversity
Evol-Instruct evolved	15–25%	Difficulty calibration
Verified math/code chains	10–20%	Reasoning capability
Orca CoT augmented	10–20%	Step-by-step process learning
Human-curated anchor	20–30%	Distribution grounding, collapse prevention

Table 5.1: Recommended SFT data mix composition using synthetic sources

Part III

Pre-training

Chapter 6

Pre-training Objectives and Strategies

Contents

5.1 The Case for Synthetic Data 17
5.2 Core Techniques 18
5.3 Quality Filtering 19
5.4 Risks: Model Collapse and Bias Amplification 19
5.5 Synthetic Data Composition Recommendations 20

What You Will Learn

- Master the next-token prediction (NTP) objective and its 2026 variants.
- Review Reinforcement Pre-Training (RPT) for enhanced reasoning.
- Analyze curriculum learning and instruction-augmented pre-training.
- Understand the multi-stage compute-optimal context extension pipeline.

6.1 Next-Token Prediction

Self-supervised causal language modeling (see also §G.2.1 for the full NTP objective and §G.2.2 for perplexity (Appendix G.2); code: Listing G.2):

$$\mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_1, \ldots, x_{t-1})$$

No labeled data needed. Captures syntax, semantics, factual knowledge, and reasoning patterns. Despite its simplicity, this objective is sufficient to produce emergent capabilities at scale.

6.2 Reinforcement Pre-Training (RPT)

Microsoft (2025): next-token prediction reframed as a sequential decision-making process. Each token prediction is a policy action; the model receives richer gradient signals than maximum likelihood. Fully self-supervised. Improves long-horizon coherence and reasoning traces.

6.3 Curriculum Learning

Data organized by difficulty (simple $\rightarrow$ complex) accelerates convergence and boosts final performance. Difficulty metrics: perplexity under a smaller reference model, task complexity (number of reasoning steps required), domain specificity.

6.4 Instruction-Response Augmented Pre-training

Synthetic instruction-response pairs in the pre-training corpus bridge the gap to SFT. Typically 1–3% of the pre-training mix. Reduces the SFT data required by 5–10$\times$ for comparable instruction-following capability.

6.5 Long-Context Pre-training

Short-context training followed by gradual context extension:

1. Train at 4K context for 90% of total compute.
2. Gradually increase to 32K–128K on a filtered long-document subset.
3. Apply YaRN or LongRoPE for positional interpolation.

Training from scratch at long context is compute-inefficient; the model wastes FLOPs on mostly short sequences in the dataset.

Chapter 7

Distributed Training and Infrastructure

Contents

6.1 Next-Token Prediction 23
6.2 Reinforcement Pre-Training (RPT) 24
6.3 Curriculum Learning 24
6.4 Instruction-Response Augmented Pre-training 24
6.5 Long-Context Pre-training 24

What You Will Learn

- Master 3D Parallelism: Data, Tensor, and Pipeline strategies for scale.
- Review ZeRO optimizer sharding and communication-efficient NCCL patterns.
- Implement mixed-precision (BF16/FP8) and fault-tolerant training.
- Evaluate the 2026 frontier training frameworks (Megatron, DeepSpeed v2).

7.1 Parallelism Strategies

- **Data Parallelism:** FSDP and DeepSpeed ZeRO shard optimizer states, gradients, and parameters.
- **Tensor Parallelism:** Split individual layers across GPUs (Megatron-LM). Column/row-parallel linear layers with all-reduce at boundaries.
- **Pipeline Parallelism:** Distribute model layers with micro-batching. The 1F1B schedule minimizes bubble overhead to $\frac{p-1}{p \cdot m}$ where p = pipeline stages, m = micro-batches.
- **Expert Parallelism:** MoE-specific expert distribution across devices.
- **Sequence Parallelism:** Ring Attention for distributed long-context attention.

7.1.1 3D Parallelism

Combining data, tensor, and pipeline parallelism simultaneously. Used for training models beyond 70B parameters:

- **Inner (intra-node):** Tensor parallelism exploiting NVLink bandwidth (8–16 GPUs).
- **Middle (inter-node):** Pipeline parallelism across nodes.
- **Outer:** Data parallelism across replica groups.

DeepSpeed 3D parallelism + ZeRO-3 achieved 95% GPU utilization on 1,024 A100s for a 175B model.

7.2 Mixed-Precision Training

BF16 (dominant, same exponent range as FP32) and FP8 (emerging frontier, native H100/H200 support). FP8 requires careful per-tensor or per-row scaling factors to prevent underflow and overflow.

7.3 Checkpointing and Fault Tolerance

Periodic snapshots, asynchronous checkpointing, distributed sharding, automatic restart/recovery. At 1,000+ GPU scale, mean time between failures of $\sim$3 days makes fault tolerance critical. Async checkpointing overlaps saving with training at near-zero overhead.

7.4 Communication Optimization

- **NVLink:** 600–900 GB/s bidirectional within a node (vs. 32 GB/s PCIe). Essential for tensor parallelism.
- **InfiniBand (400G):** 50–100 GB/s cross-node. All-reduce is the dominant communication pattern.
- **Gradient compression:** Top-k sparsification or quantized gradients for bandwidth-limited setups.

7.5 Optimizers and Learning Rate Schedules

Full update rules in Appendix G.6: AdamW, Lion, cosine schedule; code: Listing G.12, Listing G.14.

7.5.1 Optimizer Selection

- **AdamW (Loshchilov and Hutter 2019):** The 2025–2026 default. Adam with decoupled weight decay. Hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay $= 0.1$ (higher than the original 0.01 recommendation for LLM pre-training). Decoupled weight decay (unlike L2 regularization) correctly regularizes adaptive learning rates.

- **Muon (Momentum + Orthogonalization) (Kosson et al. 2024):** Applies a Nesterov momentum update followed by orthogonalization via Newton-Schulz iterations. It treats weight matrices as the correct unit of analysis rather than individual scalars, ensuring that all directions in weight space are explored with equal intensity. Matches or exceeds AdamW on pre-training perplexity with 25% fewer FLOPs on the optimizer step (see formulation §G.6.2, code Listing G.13).
- **Lion (EvoLved Sign Momentum) (Chen et al. 2023):** Sign-based update rule: uses only the sign of the gradient momentum, not its magnitude. 2–3× memory reduction vs. AdamW (no second moment). Effective learning rate typically needs to be 3–10× smaller than AdamW. Works well for fine-tuning; mixed results at large pre-training scale.
- **Sophia (Liu et al. 2023):** Diagonal Hessian preconditioned optimizer. Estimates curvature via Hutchinson estimator every 10 steps. Reported 2× faster than AdamW on pre-training; not yet widely adopted due to implementation complexity.

7.5.2 Learning Rate Schedule

Cosine decay with linear warmup is the standard:

$$\eta(t) = \eta_{\min} + \frac{1}{2}\left(\eta_{\max} - \eta_{\min}\right)\left(1 + \cos\left(\frac{t - t_{\text{warmup}}}{T - t_{\text{warmup}}}\pi\right)\right)$$

- **Warmup steps:** 1,000–5,000 tokens. Prevents gradient explosions from random initial parameter distributions.
- **Peak LR:** 3×10^{-4} (7B), 1×10^{-4} (70B). Larger models require smaller peak LR because parameter interactions are stronger.
- **Final LR:** $\eta_{\min} = 0.1 \times \eta_{\max}$ (10% of peak). Do not decay to zero: residual LR prevents overfitting on the tail of training.
- **WSD (Warmup-Stable-Decay):** Used by MiniCPM and Qwen. Three-phase schedule: warmup → constant "stable" phase → rapid decay. Enables model checkpointing at multiple scales without performance cliff.

7.5.3 Gradient Clipping and Stability

- **Gradient norm clipping:** Clip global gradient norm to 1.0. Prevents occasional spike tokens from destabilizing training. Empirically, more than 1% of steps exceeding the clip threshold indicates a hyperparameter problem.
- **Loss spike detection:** Monitor loss every 100 steps. A spike $> 3\times$ the rolling average often indicates corrupted data, a bad batch, or a numerical instability. Automated checkpointing allows rollback.
- **Batch size scaling:** Linear scaling rule: if batch size doubles, multiply LR by $\sqrt{2}$ (square root rule is more conservative than linear for LLMs). Use gradient accumulation to simulate large effective batch sizes without increasing per-device memory.

Optimizer	Memory	Speed	Best For
AdamW	3× model	Standard	Universal default
Muon	2× model	+10–25%	Pre-training, large LR
Lion	2× model	+5–15%	Fine-tuning, memory-constrained
Sophia	4× model	+80–120%	Research, fast convergence

Table 7.1: Optimizer memory footprint and speed comparison (relative to model size)

7.6 Key Training Frameworks

Framework	Primary Use	Key Features
Megatron-LM	Large-scale pre-training	Tensor/pipeline/sequence parallelism
DeepSpeed	Distributed training	ZeRO optimizer, pipeline parallelism
FSDP (PyTorch)	Data-parallel training	Integrated PyTorch ecosystem
NeMo (NVIDIA)	End-to-end platform	Pre-training to deployment
Axolotl	Fine-tuning workflows	LoRA, QLoRA, full fine-tuning
Unsloth	Memory-efficient tuning	2–5× faster, 80% less memory
TRL (Hugging Face)	Alignment training	PPO, DPO, GRPO trainers (v0.28+)

Table 7.2: Key training frameworks in 2025–2026

Part IV

Post-Training – From Base Model to Product

Chapter 8

Supervised Fine-Tuning (SFT)

Contents

7.1 Parallelism Strategies 25
7.2 Mixed-Precision Training 26
7.3 Checkpointing and Fault Tolerance 26
7.4 Communication Optimization 26
7.5 Optimizers and Learning Rate Schedules 26
7.6 Key Training Frameworks 28

What You Will Learn

- Master instruction tuning requirements and token-masking techniques.
- Implement parameter-efficient fine-tuning (PEFT) with LoRA and QLoRA.
- Analyze the transition to DoRA and rank-adaptive PEFT in 2026.
- Understand the 2025 shift from large instruction sets to high-quality "Less Is More" (LIMA) datasets.

8.1 Purpose and Principles

SFT transitions the base model to an instruction-following assistant. It is stable, inexpensive, and immediately effective—but cannot resolve preference trade-offs or handle long-tail failures alone.

8.2 Data Quality for SFT

The Llama 2 team (Touvron et al. 2023) found that 27K high-quality examples outperform 1M noisy examples. Key quality dimensions:

- **Diversity:** Cover the full distribution of user intents.
- **Accuracy:** Ground-truth responses must be verifiably correct.
- **Format compliance:** Consistent chat templates and proper system prompts.
- **Difficulty calibration:** Include challenging multi-step tasks; trivially easy examples waste model capacity.

Quality filtering approaches: LLM-as-judge scoring, perplexity-based selection, human annotation of a seed set, IFD (Instruction Following Difficulty) scoring.

8.3 Chat Templates

Chat templates define how multi-turn conversations are serialized into token sequences. Consistent templates between training and inference are critical:

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
You are a helpful assistant.<|eot_id|>
<|start_header_id|>user<|end_header_id|>
What is the capital of France?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Listing 8.1: Example chat template (Llama 3 format)

Loss masking: compute cross-entropy loss *only* on assistant tokens, not on system or user tokens. Failure to mask causes the model to learn to generate user messages.

8.4 Parameter-Efficient Fine-Tuning (PEFT)

Full derivations in Appendix G.7: LoRA (§G.7.1), DoRA (§G.7.2); code: Listing G.15.

- **LoRA (Hu et al. 2022):** Low-rank matrices on frozen weights. <1% extra parameters. Dominant PEFT method.
- **QLoRA (Dettmers et al. 2023):** LoRA + 4-bit quantization. Enables 70B+ models on a single consumer GPU.
- **DoRA (Liu et al. 2024):** Weight-Decomposed Low-Rank Adaptation. Decomposes updates into magnitude and direction components—matches or exceeds full fine-tuning on several benchmarks.
- **Spectrum:** Signal-to-noise ratio analysis identifies the most informative layers for fine-tuning, enabling partial fine-tuning with better ROI per updated parameter.

Chapter 9

Alignment – RLHF, DPO, and Beyond

Contents

8.1 Purpose and Principles 31
8.2 Data Quality for SFT 31
8.3 Chat Templates 32
8.4 Parameter-Efficient Fine-Tuning (PEFT) 32

What You Will Learn

- Trace the evolution from RLHF to reference-free methods (DPO, SimPO).
- Implement alignment with binary feedback (KTO) and group-relative rewards (GRPO).
- Evaluate the role of verifiable rewards (RLVR) in reasoning-heavy models.
- Understand the 2026 modular alignment stack for safety and helpfulness.

9.1 The Alignment Problem

A grammatically perfect model may be unhelpful, evasive, or unsafe. The field has evolved from RLHF alone to a modular stack of complementary methods.

9.2 Reward Modeling

Complete objectives in Appendix G.8: PPO, DPO, SimPO, KTO, GRPO.

The reward model (RM) translates human preferences into scalar scores. RM quality determines the ceiling for all RM-based alignment methods.

- **Architecture:** Language model with a regression head replacing the LM head.

- **Training:** Bradley-Terry model on pairwise comparisons (Ziegler et al. 2019): $p(y_w \succ y_l) = \sigma(r(x, y_w) - r(x, y_l))$.
- **Reward hacking:** An RM optimized too strongly diverges from true human preferences. KL penalty and early stopping mitigate this.
- **Process Reward Models (PRM) (Lightman et al. 2023):** Score each intermediate reasoning step, not just the final output. More signal, less sparse. Used in OpenAI's o-series for math and coding.

9.3 RLHF

Transformed GPT-3 into ChatGPT (Ouyang et al. 2022). Pipeline: SFT → Reward Model → PPO (Schulman et al. 2017). Essential for open-ended quality but requires four models in memory simultaneously and is prone to reward hacking.

9.4 DPO

Eliminates the reward model entirely (Rafailov et al. 2023) (formula §G.8.1, code Listing G.17). 90–95% of RLHF performance at 40–60% less compute.

DPO Successors

- **SimPO (Meng et al. 2024):** Eliminates the reference model entirely. It uses length-normalized log probabilities as an implicit reward and introduces a target reward margin γ (see formulation §G.8.2, code Listing G.18).
- **KTO (Kahneman-Tversky Optimization) (Ethayarajh et al. 2024):** Operates on binary thumbs-up/down signals rather than preferences. It maximizes the utility of model outputs by aligning with human loss aversion (see formulation §G.8.4, code Listing G.20).
- **Iterative DPO:** Semi-online re-sampling from the current policy. Closes the distribution gap between training data and current model outputs.
- **IPO (Azar et al. 2024):** Information-Preference Optimization. Regularizes DPO to prevent overfitting to preference margins.

9.5 GRPO

DeepSeek's dominant algorithm for reasoning (Guo et al. 2025) (formula §G.8.3, code Listing G.19). Generates a group of 8–64 responses per prompt. Advantages normalized against group statistics. Eliminates the critic model, yielding 33–50% memory savings over PPO.

$$\mathcal{L}_{\text{GRPO}} = \mathbb{E}\left[\sum_i \hat{A}_i \cdot \log \pi_\theta(o_i \mid q) - \beta \cdot \text{KL}[\pi_\theta \| \pi_{\text{ref}}]\right]$$

where $\hat{A}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$ is the group-normalized advantage.

9.6 DAPO

Removes reference model dependency from GRPO (Yu et al. 2025). The most streamlined RL alignment approach. Also introduces *clip-higher* (asymmetric policy clipping) and token-level loss normalization to stabilize training.

9.7 RLAIF and Constitutional AI

AI-generated preferences replace human annotations. Anthropic's CAI (Bai et al. 2022): model self-critiques against constitutional principles, creating scalable alignment data without per-example human annotation.

9.8 RLVR

Verifiable rewards for math and code. Binary, objective signal resists reward hacking. DeepSeek R1 demonstrated emergent reasoning from pure RLVR + GRPO without SFT. The key insight: verifiable correctness is a cleaner training signal than a learned reward model.

9.9 The Modern Alignment Stack

Stage	Technique	What It Solves
Instruction Following	SFT	Format, tone, task execution
Preference Alignment	DPO / SimPO / KTO	Human values, norms
Reasoning	GRPO / DAPO + RLVR	Math, code, planning
Safety & Helpfulness	RLHF (neural RM)	Open-ended quality, harm avoidance
Constitutional Guardrails	CAI / RLAIF	Scalable safety principles

Table 9.1: The modular alignment stack in 2026

Chapter 10

Training for Reasoning

Contents

9.1 The Alignment Problem 33
9.2 Reward Modeling 33
9.3 RLHF 34
9.4 DPO 34
9.5 GRPO 34
9.6 DAPO 35
9.7 RLAIF and Constitutional AI 35
9.8 RLVR 35
9.9 The Modern Alignment Stack 35

What You Will Learn

- Analyze Chain-of-Thought (CoT) and its 2025–2026 training methods.
- Master Process Reward Models (PRM) for step-level supervision.
- Implement test-time compute scaling and search (MCTS, Best-of-N).
- Understand Thinking Tokens (Quiet-STaR) for emergent reasoning.

10.1 The Rise of Reasoning Models

Models that break complex problems into explicit intermediate steps—the most significant capability development of 2025–2026. Reasoning models sacrifice token efficiency for accuracy on hard tasks.

10.2 Chain-of-Thought (CoT)

Prompting or training models to generate intermediate reasoning steps before the final answer. Few-shot CoT (Wei et al. 2022) demonstrated step-by-step prompting dramatically improves multi-step arithmetic, symbolic reasoning, and commonsense tasks.

- **Zero-shot CoT:** Append "Let's think step by step" to the prompt.
- **Few-shot CoT:** Provide example (question, reasoning chain, answer) triples in the prompt.
- **Self-consistency:** Sample k CoT paths and take the majority vote answer. +5–15% on GSM8K vs. single-sample CoT.

10.3 Process Reward Models (PRM)

Instead of rewarding only the final answer, PRMs assign a score to each reasoning step:

- Denser supervision signal: n steps $\rightarrow$ n reward signals per example.
- Enables early pruning of incorrect branches during tree search.
- Used in OpenAI's o-series for competition math and coding (Lightman et al. 2023).

Annotation challenge: step-level labels are expensive. Solutions: automated labeling via outcome consistency (steps on paths leading to correct answers are labeled positive) and synthetic step decomposition from a stronger model.

10.4 Test-Time Search

Allocate more inference compute per query to find better solutions:

- **Best-of-N:** Sample N complete solutions; select via outcome RM or verifier.
- **Beam Search over CoT:** Expand k partial chains at each step; prune with PRM.
- **Monte Carlo Tree Search (MCTS):** Full tree exploration with PRM scoring. Used in AlphaCode 2 and OpenAI o3.

Test-Time Compute Scaling

Snell et al. (2024) showed that test-time compute can match training-time compute scaling: a model with 16× more test-time compute can match a model 16× larger. This creates a new performance axis independent of parameter count.

10.5 TLT – Accelerating Training

MIT: a smaller "drafter" model predicts outputs; a larger verifier model confirms them (Leviathan et al. 2023). 70–210% acceleration with preserved accuracy. The drafter is reusable for speculative decoding at inference.

10.6 R1-Style Pure RL Training

DeepSeek demonstrated that pure RL on verifiable tasks produces emergent reasoning without any SFT (Guo et al. 2025):

1. Start from the base model (no instruction tuning).
2. Apply GRPO (formulation §G.8.3) with binary correctness reward on math/code tasks.
3. Emergent behaviors: self-verification ("wait, let me reconsider"), extended thinking, strategy switching.

Pure RL Training Instability

R1-Zero (pure RL, no SFT cold start) exhibits language mixing, repetitive patterns, and poor readability. The DeepSeek production pipeline uses a cold-start SFT phase (a few thousand high-quality CoT examples) before GRPO to stabilize training.

10.7 Thinking Tokens (Quiet-STaR)

Rather than forcing reasoning into a separate pre-response phase, Quiet-STaR (Zelikman et al. 2024) trains the model to generate "internal monologue" tokens at every position in the sequence:

- **Divergent Thinking:** The model generates multiple parallel reasoning traces (thoughts) in the background.
- **Rationalization:** It selects the thought that most improves the prediction of the subsequent actual text tokens.
- **Token-Level RL:** Thoughts are rewarded based on how much they reduce the cross-entropy loss of the next real token.

This allows models to benefit from reasoning even on tasks where explicit Chain-of-Thought is not requested, and it creates a unified architecture for both "fast" (direct) and "slow" (reasoned) processing.

The Thinking Token Trade-off

Thinking tokens increase inference compute (latency and FLOPs) but can dramatically reduce the model size required to achieve a certain level of performance. A 7B model with 16 thinking tokens can match a 70B model on several reasoning-heavy benchmarks.

Chapter 11

Knowledge Distillation and Model Compression

Contents

10.1 The Rise of Reasoning Models **37**
10.2 Chain-of-Thought (CoT) **38**
10.3 Process Reward Models (PRM) **38**
10.4 Test-Time Search **38**
10.5 TLT – Accelerating Training **38**
10.6 R1-Style Pure RL Training **39**
10.7 Thinking Tokens (Quiet-STaR) **39**

What You Will Learn

- Master the forward and reverse KL distillation loss functions.
- Implement sequence-level distillation for complex reasoning tasks.
- Analyze the trade-offs between student model size and knowledge retention.
- Evaluate the role of speculative decoding as a dynamic distillation strategy.

Knowledge distillation (KD) transfers capability from a large *teacher* model to a smaller *student*, achieving performance beyond what the student could reach by training on ground-truth labels alone.

11.1 Why Distillation Matters

Complete derivations in Appendix G.9: forward KL, reverse KL (MiniLLM), speculative decoding acceptance; code: Listing G.21, Listing G.22.

Frontier labs use distillation systematically: Behemoth → Scout/Maverick (Meta), R1-671B → 1.5B–70B (DeepSeek), Gemini Pro → Flash → Nano (Google). Distilled 7B models now routinely outperform undistilled 70B models on target tasks.

11.2 Response (Output-Level) Distillation

The simplest form: generate responses from the teacher and use them as SFT labels for the student. The student minimizes the negative log-likelihood of teacher completions rather than human-annotated completions.

$$\mathcal{L}_{\text{resp}} = -\sum_t \log p_\theta(y_t^{\text{teacher}} \mid x, y_{<t}^{\text{teacher}})$$

Advantages: Simple pipeline; works with closed-source teachers (you only need teacher outputs, not logits). **Disadvantage:** Ignores the full teacher probability distribution—the student does not learn "how uncertain" the teacher is.

11.3 Logit-Level Distillation

When teacher logits are available (open-weight teachers), the student matches the teacher's full output distribution via KL divergence:

$$\mathcal{L}_{\text{KD}} = \alpha \cdot \mathcal{L}_{\text{CE}}(y^{\text{gt}}, p_\theta) + (1-\alpha) \cdot T^2 \cdot D_{\text{KL}}\Big(\text{softmax}(z^{\text{teacher}}/T) \parallel \text{softmax}(z^{\text{student}}/T)\Big)$$

where T is the temperature (commonly 2–4) and α balances ground-truth and teacher loss. Higher temperature softens the teacher distribution, providing richer signal on near-correct tokens.

11.4 Sequence-Level Knowledge Distillation (SeqKD)

Rather than optimizing token-level KL, SeqKD (Kim and Rush 2016) generates *mode sequences* from the teacher (beam search, temperature = 0) and treats them as hard labels. More cache-friendly than online logit distillation; works without access to logits. Used extensively when distilling from GPT-4 or Claude.

11.5 MiniLLM: Reverse KL for Distillation

Standard KD minimizes forward KL $D_{\text{KL}}(p_{\text{teacher}} \| p_{\text{student}})$, which forces the student to cover all teacher modes and leads to "mean-seeking" behaviour. MiniLLM (Gu et al. 2024) minimizes reverse KL $D_{\text{KL}}(p_{\text{student}} \| p_{\text{teacher}})$, encouraging mode-seeking: the student becomes sharp and accurate on the teacher's most probable outputs, avoiding the diffusion of probability mass across irrelevant alternatives. Reverse KL is estimated via policy gradient on student-generated sequences with teacher log-probability as reward.

11.6 Intermediate Layer Distillation

Align hidden states or attention patterns at intermediate layers, not just the output:

- **Feature-map distillation (PKD, TinyBERT):** $\mathcal{L}_{\text{feat}} = \|W_{\text{proj}} h_l^{\text{student}} - h_{f(l)}^{\text{teacher}}\|_2^2$ where $f(l)$ maps student layer to teacher layer index and W_{proj} is a learned projection.
- **Attention-pattern distillation:** $\mathcal{L}_{\text{attn}} = D_{\text{KL}}(A_{l,h}^{\text{teacher}} \| A_{l,h}^{\text{student}})$ summed over heads and layers.
- **Practical note:** Intermediate distillation requires matching architectures or learned projections. Most practical pipelines stick to output-level or logit-level distillation.

11.7 On-Policy Distillation

A key failure mode of offline response distillation: the student is trained on teacher trajectories but evaluated on *its own* trajectories. Distribution mismatch grows with model capability gap. On-policy distillation:

1. Roll out the *student* to generate candidate completions.
2. Score each completion with the teacher (assign log-probability or RM score).
3. Minimize KL between student and teacher-scored distribution.

Requires either white-box access to teacher logits or a fast teacher inference endpoint. Used in GKD (Agarwal et al. 2024) (Generalized Knowledge Distillation).

11.8 Speculative Decoding as Distillation

Speculative decoding (Leviathan et al. 2023) uses a small draft model to propose k tokens, verified in parallel by the target model. Beyond a decoding speedup, training the drafter to maximally predict acceptance by the target is a form of distillation—the drafter learns to approximate the target's conditional distribution on easy tokens, which constitute 70–90% of all tokens.

11.9 Practical Distillation Workflow

1. Select teacher (strongest accessible model: open-weight 70B–671B or API).
2. Generate responses on your instruction set. For reasoning tasks, generate with high temperature ($T = 0.7$–1.0) and filter by verifiable correctness.
3. Optionally: collect teacher logits for logit-level KD (requires local teacher).
4. Fine-tune student with $\alpha = 0.5$: 50% KD loss, 50% standard CE loss on human labels.
5. Evaluate on target benchmarks; iterate on temperature and α.

Distillation Rules of Thumb

- A 7B student distilled from a 70B teacher typically closes 60–80% of the gap to the teacher.
- Distillation data volume: 50K–500K high-quality teacher completions is sufficient for most fine-tuning objectives.
- For reasoning tasks, only keep teacher traces that lead to correct verified answers—incorrect reasoning chains hurt more than they help.
- On-policy distillation adds significant complexity; prefer offline SeqKD unless you observe severe distribution mismatch.

```
import torch
import torch.nn.functional as F

def distillation_loss(student_logits, teacher_logits,
                      labels, alpha=0.5, temperature=2.0):
    """
    Combined cross-entropy + KL distillation loss.
    student_logits, teacher_logits: (batch, seq_len, vocab)
    labels: (batch, seq_len)  -100 for masked tokens
    """
    # Standard cross-entropy on ground-truth labels
    ce_loss = F.cross_entropy(
        student_logits.view(-1, student_logits.size(-1)),
        labels.view(-1),
        ignore_index=-100,
    )

    # KL divergence on teacher distribution
    soft_teacher = F.softmax(teacher_logits / temperature, dim=-1)
    soft_student = F.log_softmax(student_logits / temperature, dim
   =-1)
    kd_loss = F.kl_div(
        soft_student, soft_teacher,
        reduction="batchmean", log_target=False,
    ) * (temperature ** 2)

    return alpha * ce_loss + (1 - alpha) * kd_loss
```

Listing 11.1: Logit-level knowledge distillation training step

Part V

Evaluation, Safety & Deployment

Chapter 12

Evaluating LLMs – Benchmarks and Beyond

Contents

11.1 Why Distillation Matters 41

11.2 Response (Output-Level) Distillation 42

11.3 Logit-Level Distillation 42

11.4 Sequence-Level Knowledge Distillation (SeqKD) 42

11.5 MiniLLM: Reverse KL for Distillation 42

11.6 Intermediate Layer Distillation 43

11.7 On-Policy Distillation 43

11.8 Speculative Decoding as Distillation 43

11.9 Practical Distillation Workflow 43

What You Will Learn

- Master the 2026 benchmark suite: MMLU-Pro, HumanEval, and GPQA.
- Implement LLM-as-a-Judge and preference-based ELO evaluation systems.
- Detect and mitigate data contamination in benchmark results.
- Evaluate qualitative capabilities through red-teaming and side-by-side comparison.

Benchmark	What It Measures	Type
MMLU / MMLU-Pro	Broad academic knowledge (57+ subjects)	Multiple choice
HumanEval / MBPP	Code generation correctness	Code execution
GSM8K / MATH	Mathematical problem solving	Verified answers
TruthfulQA	Resistance to misconceptions	Open-ended
MT-Bench	Multi-turn conversation	LLM-judged
AlpacaEval 2	Instruction-following quality	LLM-judged
Arena-Hard	Head-to-head comparison	LLM-judged
SWE-bench	Real-world software engineering	Code execution
AIME 2024	Competition mathematics	Verified answers
GPQA-Diamond	Expert-level science reasoning	Multiple choice

Table 12.1: Standard LLM evaluation benchmarks

12.1 LLM-as-Judge

LLM judges (GPT-4, Claude, Gemini) score open-ended responses. Pitfalls: position bias (judges prefer the first response), verbosity bias (longer responses rated higher), self-enhancement bias (models rate their own outputs higher). Mitigations: swap positions across evaluations, length-normalize scores, use multiple judge models.

12.2 Chatbot Arena and ELO Rankings

Chatbot Arena (LMSYS) collects millions of pairwise human comparisons and computes ELO ratings. Statistically robust due to scale; reflects real user preferences better than academic benchmarks. High ELO and high MMLU are different things—they measure different aspects of model quality.

12.3 Benchmark Contamination and Gaming

- Evaluation data present in pre-training inflates scores. Always report decontamination procedures.
- Optimizing specifically for a benchmark can improve scores without improving underlying capability (Goodhart's Law).
- Mitigations: dynamic benchmarks with new questions each run, held-out test sets never used for selection, diverse multi-benchmark suites that resist narrow optimization.

Chapter 13

Safety, Ethics, and Constitutional AI

Contents

12.1 LLM-as-Judge 48
12.2 Chatbot Arena and ELO Rankings 48
12.3 Benchmark Contamination and Gaming 48

What You Will Learn

- Implement red-teaming and adversarial testing for "jailbreak" prevention.
- Master Constitutional AI and the RLAIF (RL from AI Feedback) stack.
- Analyze mitigation strategies for bias, hallucination, and PII leakage.
- Review the 2026 safety standards for high-stakes AI deployments.

13.1 Safety Taxonomy

LLM safety failures fall into three broad categories:

1. **Harmfulness:** Generating dangerous, illegal, or offensive content.
2. **Dishonesty:** Hallucinating facts, sycophantic agreement, deceptive framing.
3. **Misalignment:** Pursuing goals misaligned with user or societal intent.

13.2 Constitutional AI (CAI)

Anthropic's Constitutional AI provides a scalable alternative to extensive human annotation:

1. **SL-CAI:** For each harmful response, the model generates a critique against a written constitution, then revises the response. Revised outputs become SFT training data.

2. **RL-CAI:** AI-generated preference labels (RLAIF) replace human annotators for training the reward model. Constitutional principles guide preference generation.

Advantages: dramatically reduces human annotation cost; the constitution is auditable and adjustable; generalizes to unseen harm categories.

13.3 Red-Teaming

- **Manual red-teaming:** Human adversarial testers find novel jailbreaks and failure modes that automated methods miss.
- **Automated red-teaming:** LLM generates adversarial prompts at scale (HarmBench, AdvBench). Identifies systematic weaknesses efficiently.
- **Structured output red-teaming:** Test tool-use, JSON outputs, and code execution paths for injection vulnerabilities.

13.4 Interpretability and Explainability

SHAP and LIME for feature attribution. Attention visualization (with caveats—attention patterns are not explanations). Mechanistic interpretability (Anthropic): identify circuits and features responsible for specific behaviors. EU AI Act (2024–2026) mandates transparency for high-risk AI applications.

13.5 Safe Deployment Practices

- Output classifiers (toxicity, PII detection) as guardrails.
- Input filtering for known harmful patterns.
- Rate limiting and usage monitoring.
- Human-in-the-loop for high-stakes decisions.
- Staged rollouts with safety monitoring before broad deployment.

Chapter 14

Inference and Deployment at Scale

Contents

13.1 Safety Taxonomy 49

13.2 Constitutional AI (CAI) 49

13.3 Red-Teaming 50

13.4 Interpretability and Explainability 50

13.5 Safe Deployment Practices 50

What You Will Learn

- Distinguish between throughput-bound and memory-bound inference.
- Master KV cache management (PagedAttention, RadixAttention).
- Implement the 2026 quantization stack (H100 FP8, post-training quantization).
- Optimize throughput with speculative decoding and drafting.

14.1 Quantization

Format	Size Reduction	Quality Loss	Best For
FP16 / BF16	2×	Minimal	Training
INT8 (LLM.int8(Dettmer et al. 2022))	∼4×	Negligible	Serving large models
INT4 / GPTQ / AWQ	∼8×	Small	Throughput-optimized serving
GGUF (Q4_K_M)	∼6×	Small	Local / edge inference
FP8	4×	Minimal	H100/H200 native serving
1.58-bit (BitNet)	∼20×	Moderate	Research / edge

Table 14.1: Quantization formats and trade-offs

14.2 Inference Frameworks

vLLM (PagedAttention), TGI (Hugging Face), TensorRT-LLM (NVIDIA), llama.cpp/Llamafile (local/edge).

14.3 Speculative Decoding

A small draft model generates k tokens; the large target model verifies all k in parallel (Leviathan et al. 2023). 2–3× speedup, zero quality loss. Works best when the draft model acceptance rate is high. Medusa: multiple parallel draft heads on the target model itself, eliminating the separate draft model.

14.4 Continuous Batching

Naive batching waits for all requests in a batch to complete before accepting new ones—wasteful when requests differ in length. Continuous batching (Yu et al. 2022) inserts new requests as slots free up, achieving near-100% GPU utilization.

14.5 KV Cache Management

KV cache formula (§G.9.3), quantization bounds (§G.9.4) in Appendix G.2; speculative decoding code: Listing G.22.

- **PagedAttention (vLLM) (Kwon et al. 2023):** Non-contiguous physical KV cache blocks managed like OS virtual memory. Eliminates fragmentation, enables 2–4× more concurrent requests.

- **Prefix caching:** Cache shared system prompt KV states across requests. 50–80% cache hit rates for common deployments.
- **KV quantization:** Quantize cached KV to INT8/INT4 to extend effective context within fixed memory.

14.6 Multi-GPU Inference

For models exceeding single-GPU VRAM, use tensor parallelism at inference:

- 2–4 GPUs: tensor parallelism within a node (NVLink bandwidth sufficient).
- 8+ GPUs: tensor + pipeline parallelism.
- Reference: a 70B model at BF16 requires 4×A100-80GB; INT4 fits on 2×A100-40GB.

Part VI

Advanced Topics and Future Directions

Chapter 15

Specialized Domains and Multi-modality

Contents

14.1 Quantization 52
14.2 Inference Frameworks 52
14.3 Speculative Decoding 52
14.4 Continuous Batching 52
14.5 KV Cache Management 52
14.6 Multi-GPU Inference 53

What You Will Learn

- Master data curation and fine-tuning for Coding, Law, and Science.
- Implement the 2026 multi-modal stack: Vision, Audio, and Action.
- Analyze world models and agentic integration in specialized domains.
- Evaluate the "Small Language Model" (SLM) trend for domain-specific edge AI.

15.1 Domain-Specific Pre-training

Domain-specific models outperform general models on target tasks because domain vocabulary and reasoning patterns require exposure during pre-training, not just fine-tuning.

- **BloombergGPT (Wu et al. 2023) (finance):** 50% financial text, 50% general. Outperforms GPT-3 on financial benchmarks while remaining competitive on general tasks. Key lesson: 50/50 mix preserves generality.
- **Med-PaLM 2 (Singhal et al. 2023) (medical):** Expert-level performance on the USMLE. Multi-step reasoning over clinical evidence requires both medical knowledge and reasoning capabilities.

- **CodeLlama / DeepSeek-Coder:** Code-specialized continued pre-training from a general base. Infill (fill-in-the-middle) pre-training objective for code completion.
- **ChatLAW (legal):** Legal corpus with case law, statutes, and regulatory filings.

15.2 Multimodal Models

- **Vision-Language Models (VLM):** Connect a vision encoder (CLIP, SigLIP) to an LLM via a projection layer or cross-attention. LLaVA, InternVL, Qwen3-VL.
- **Early fusion (Llama 4):** Text and image tokens processed jointly from the first layer. More parameter-efficient at scale than late-fusion adapters.
- **Vision-Language MoE (Qwen3-VL):** Cross-modal compute allocation—vision and language tokens routed to specialized experts.
- **Omni-modal:** Audio, video, and document understanding in a unified model (Gemini 2.5 Pro, GPT-4o).

15.3 Agentic AI

LLMs as agents: perceive environment, plan actions, use tools, and produce observable effects. Training considerations:

- End-to-end agent evaluation (SWE-bench, WebArena, GAIA) requires operating in real environments, not just text.
- Tool-use fine-tuning: function calling, structured JSON output, code execution.
- Multi-agent coordination: multiple specialized LLMs operating in concert.

Chapter 16

Model Merging and Recombination

Contents

15.1 Domain-Specific Pre-training 57
15.2 Multimodal Models 58
15.3 Agentic AI 58

What You Will Learn

- Master task vector arithmetic and SLERP weight interpolation.
- Implement TIES-Merging and DARE for combining fine-tuned experts.
- Evaluate the "FrankenMoE" strategy for creating heterogeneous ensembles.
- Analyze the 2025 shift from training singular models to evolving model families.

Model merging combines the weights of multiple fine-tuned models without additional training, enabling capability combination at near-zero compute cost.

16.1 Merging Methods

- **Linear interpolation:** $\theta = (1-\alpha)\theta_A + \alpha\theta_B$. Simple but creates interference between dissimilar capabilities.
- **SLERP (Spherical Linear Interpolation):** Interpolates along the geodesic of weight space. Better than linear for preserving each model's characteristics.
- **TIES-Merging (Yadav et al. 2023):** Trim redundant deltas → Elect sign direction → Disjoint merge. Handles sign conflicts between different fine-tuned models.
- **DARE (Yu et al. 2023) (Drop And REscale):** Randomly drop task-vector parameters and rescale survivors. Reduces interference between merged models by eliminating redundant updates.

- **Model Breadcrumbs:** Remove outlier parameters before merging. Outliers cause interference; removing them improves merge quality.

16.2 Task Vectors

Full formulations in Appendix G.10: task arithmetic, SLERP, TIES, DARE; code: Listing G.24, Listing G.25.

Task vector (Ilharco et al. 2023) $\tau = \theta_{\text{fine-tuned}} - \theta_{\text{base}}$. Arithmetic on task vectors enables:

- **Capability addition:** $\theta_{\text{base}} + \tau_A + \tau_B$ combines coding and math capabilities.
- **Capability subtraction:** $\theta_{\text{base}} - \tau_{\text{toxic}}$ removes unwanted behaviors.
- **Analogy:** $\theta_{\text{base}} + \tau_{\text{French}} - \tau_{\text{English}}$ adapts a model to French.

When Merging Works Best

Merging works best when models share the same base and have been fine-tuned on complementary (not conflicting) tasks. Models fine-tuned on the same task but different data benefit more from ensemble methods than weight merging.

16.3 Practical Merging Workflow

1. Fine-tune n specialized models from the same base.
2. Compute task vectors $\tau_i = \theta_i - \theta_{\text{base}}$ for each.
3. Apply TIES or DARE to reduce interference.
4. Evaluate merged model on target benchmarks; tune merge coefficients α_i.
5. Distribute as a single merged model.

Tools: `mergekit` (Ilharco et al. 2023) (open source), Hugging Face `transformers` weight manipulation.

Chapter 17

Continual Learning and Lifelong Adjustment

Contents

16.1 Merging Methods 59
16.2 Task Vectors 60
16.3 Practical Merging Workflow 60

What You Will Learn

- Mitigate catastrophic forgetting using EWC and Low-Rank Adaptation.
- Implement dynamic weight updates for real-time knowledge injection.
- Analyze the 2026 architectures designed for memory and context retention.
- Evaluate the trade-offs between architectural stability and plasticity.

17.1 The Forgetting Problem

EWC Fisher penalty: §G.11.1 in Appendix G.11; code: Listing G.26.

When a model is fine-tuned on task B, performance on previously learned task A degrades—sometimes catastrophically. This limits the ability to update deployed models without full retraining.

17.2 Mitigation Strategies

- **Elastic Weight Consolidation (EWC) (Kirkpatrick et al. 2017):** Penalizes updates to weights important for previous tasks, identified via the Fisher information matrix approximation.

- **LoRA-based updates:** Fine-tune only low-rank adapter weights; base model weights remain unchanged. Task-specific adapters can be swapped at inference time.
- **Replay-based methods:** Mix a small fraction of old task data into new training. 5% replay often sufficient to preserve 95% of original task performance.
- **Hierarchical optimization:** MAML-like meta-learning for fast adaptation with minimal forgetting.
- **LoRA merging:** Train multiple domain-specific LoRA adapters and merge them periodically, avoiding full model retraining entirely.

17.3 Continual Pre-training

Updating the base model on new data distributions (new knowledge, new language):

- Learning rate warmup prevents early-stage gradient explosions on new data.
- Data mixing: 80% new data + 20% replay from original pre-training corpus.
- Context extension: interleave original context-length data with long-context data during extension training.

Chapter 18

The Future of LLM Training

Contents

17.1 The Forgetting Problem 61
17.2 Mitigation Strategies 61
17.3 Continual Pre-training 62

What You Will Learn

- Analyze the path towards AGI through recursive self-improvement.
- Review the scaling of world models and physical agency.
- Evaluate the 2027–2028 roadmap for energy-efficient AI research.
- Reflect on the changing role of the human engineer in the age of Agentic AI.

18.1 Dominant Trends

- **Post-training dominance:** A strong base model plus well-designed post-training can match models 10× larger with less careful alignment.
- **Reasoning as a trainable skill:** RLVR + GRPO enable systematic reasoning improvements beyond what SFT achieves.
- **Synthetic data at scale:** Frontier models generate training data for the next generation. Data flywheel dynamics favor incumbents with strong base models.
- **Efficiency breakthroughs:** MoE, FP8, speculative decoding, and quantization collectively enable 10–50× inference cost reduction vs. 2023 baselines.
- **Open-source convergence:** Open models (Llama, Qwen, Mistral) approach closed model performance within 6–12 months of release.

- **Test-time compute scaling:** A new scaling axis complementing parameter and data scaling, trading inference cost for accuracy.

18.2 Open Challenges

- **Data scarcity:** Web text quality has plateaued; synthetic data introduces distribution shift and mode collapse risks.
- **Evaluation gaps:** No reliable benchmark for AGI-level tasks, agentic behavior, or long-horizon planning.
- **Safety at scale:** Alignment techniques that work at 7B may not generalize to 1T+ parameter models.
- **Interpretability:** We cannot reliably predict emergent capabilities or explain failure modes.
- **Energy costs:** Frontier training runs consume megawatt-hours; inference at scale requires dedicated data center planning.
- **Legal and regulatory:** EU AI Act, copyright litigation, and data provenance requirements create growing compliance burdens.
- **Multimodal grounding:** Models that understand the physical world through vision, audio, and action remain an open research frontier.

18.3 The Next Frontier: 2026–2028

Likely near-term directions: hardware-software co-design (custom AI chips with in-memory compute), world models for physical reasoning, neurosymbolic integration for reliable formal reasoning, and on-device continual learning for personalization without data privacy risks.

Chapter 19

Case Studies and Frontier Lab Analysis

Contents

18.1 Dominant Trends 63
18.2 Open Challenges 64
18.3 The Next Frontier: 2026–2028 64

What You Will Learn

- Analyze the architectural and data choices of Llama 4, GPT-5, and DeepSeek-V4.
- Compare the alignment strategies of Anthropic (Constitutional AI) vs. OpenAI.
- Review the open-weights development landscape at the 400B+ scale.
- Reflect on the hardware-software co-design trends at Google, NVIDIA, and Meta.

19.1 OpenAI – GPT-5 Family

OpenAI – GPT-5 / o3 / o4-mini

Architecture: Dense transformer (rumored MoE), Azure AI supercomputers. Unified intelligence, reasoning, coding, multimodality.
Pre-training: Massive-scale unsupervised learning. GPT-4.5: data from smaller models. GPT-5: 94.6% AIME, 74.9% SWE-bench, 84.2% MMMU.
Post-training: SFT + RLHF + "safe completions" (output-centric safety). Reasoning via dedicated RL chains of thought. Process reward models score intermediate reasoning steps.
Deployment: Real-time router (instant vs. deep reasoning). ∼80% fewer factual errors, 50–80% fewer output tokens.

19.2 DeepSeek – R1 and V3

DeepSeek – R1 / V3

Architecture: Fine-grained MoE, shared expert isolation, multi-head latent attention (MLA).
Pre-training: Trillions of tokens at ∼$5.5M.
R1 pipeline (Guo et al. 2025) (4 stages):

1. Cold-Start SFT (thousands of examples)
2. RLVR + GRPO (16 responses/prompt, rule-based rewards, no neural RM)
3. Rejection Sampling + SFT Stage 2 (800K filtered examples)
4. Final RL: RLVR (reasoning) + RLHF (helpfulness/safety with separate RMs)

Results: R1-Zero AIME: 15.6% → 71.0% via pure RL. Emergent self-verification, "aha moments."
Distillation: 1.5B–70B models. 7B: 55.5% AIME 2024.

19.3 Meta – Llama 4 Family

Meta – Llama 4 Scout / Maverick / Behemoth

Architecture: First Llama MoE. Scout: 17B/109B (16 exp.). Maverick: 17B/400B (128 exp.). Behemoth: 288B/∼2T. iRoPE: 10M-token context.
Pre-training: Early fusion multimodality (text + image + video from start). MetaCLIP vision encoder.
Post-training (key insight: heavy SFT hurts RL):

- Lightweight SFT: removed 50%+ easy examples (Llama-as-judge)
- Online RL: continuous, adaptive filtering, curriculum design
- Lightweight DPO: final polish
- Behemoth codistillation: teacher → Scout/Maverick

Deployment: Scout on 1×H100 (INT4). 9–23× price-performance vs. GPT-4o.

19.4 Anthropic – Claude Family

Anthropic – Claude 4 Family

Architecture: Dense transformer. Details not publicly disclosed.
Constitutional AI (CAI):

- Phase 1 (SL-CAI): Generate → self-critique against principles → revise. Revised outputs become SFT data.
- Phase 2 (RL-CAI): Revised outputs train preference model for RLHF. RLAIF reduces

human annotation needs.
Safety: Classifiers, harmlessness RMs, extensive red-teaming. Honest, harmless, helpful.

19.5 Google DeepMind – Gemini Family

Google DeepMind – Gemini 2.5

Architecture: Natively multimodal transformer on TPU v5. Text, images, audio, video, code unified.
Post-training: SFT + RLHF + reasoning RL + "thinking" mode. Distillation: Pro → Flash → Nano.
Deployment: Google products, Vertex AI, on-device Nano.

19.6 Alibaba / Qwen – Qwen 3 Family

Alibaba – Qwen 3 / Qwen 3-VL

Architecture: Dense + MoE. Qwen3-VL: vision-language MoE (30B-A3B, 235B-A22B).
Post-training: GSPO (Group Sequence Policy Optimization). "Thinking" toggle.
Open ecosystem: Full open weights 0.6B–235B. Popular base for R1 distillation.

19.7 Cross-Cutting Patterns

Converging Trends Across Frontier Labs

- **Modular post-training:** Distinct stages, not monolithic. Heavy SFT hurts RL (Meta).
- **MoE is default:** Meta, DeepSeek, Qwen, rumored closed labs.
- **GRPO displaces PPO:** Critic-free, 33–50% memory savings.
- **Distillation is first-class:** Behemoth→Scout, R1→1.5B–70B, Pro→Flash→Nano.
- **Reasoning via RL:** Pure RL (DeepSeek), unified routing (OpenAI), thinking modes (Google/Qwen).
- **Safety diverges productively:** Safe completions, CAI, Llama-as-judge, separate RMs.

Company	Arch.	Post-Training Pipeline	Key Innovation
OpenAI	Dense (MoE?)	SFT → Safe Compl. → RLHF → Reasoning RL	Safe completions
DeepSeek	Fine MoE	Cold SFT → GRPO → Rej. Samp. → SFT2 → RLHF+RLVR	Pure RL reasoning
Meta	MoE (128 exp.)	Light SFT → Online RL → Light DPO	Light SFT + heavy RL
Anthropic	Dense	SFT → CAI → RLHF/RLAIF	Constitutional AI
Google	Multimodal	SFT → RLHF → Reas. RL → Distill.	Native multimodality
Qwen	Dense+MoE	SFT → GSPO → Reasoning RL	GSPO, multilingual

Table 19.1: Comparison of frontier lab training approaches

Appendix A

Glossary of Key Terms

RLHF Reinforcement Learning from Human Feedback – Alignment using human preference data and a reward model.

DPO Direct Preference Optimization – Alignment without a reward model, via classification on preference pairs.

SimPO Simple Preference Optimization – DPO variant using average log probability as implicit reward; no reference model required.

IPO Identity Preference Optimization – Regularized DPO that prevents overfitting to preference margins.

KTO Kahneman-Tversky Optimization – Alignment from binary feedback (thumbs up/down).

GRPO Group Relative Policy Optimization – Critic-free RL comparing responses within a group.

DAPO Decoupled Alignment and Policy Optimization – Streamlined GRPO removing the reference model.

GSPO Group Sequence Policy Optimization – Qwen's sequence-level variant of GRPO.

RLVR Reinforcement Learning with Verifiable Rewards – Training on auto-checkable tasks (math, code).

SFT Supervised Fine-Tuning – Training on instruction-response pairs.

LoRA Low-Rank Adaptation – Parameter-efficient fine-tuning with low-rank matrices.

QLoRA Quantized LoRA – LoRA combined with 4-bit base model quantization.

DoRA Weight-Decomposed Low-Rank Adaptation – Decomposes updates into magnitude and direction components.

MoE Mixture of Experts – Sparse architecture activating a subset of parameters per token.

GQA Grouped Query Attention – Shared KV heads grouped among query heads; reduces KV cache size.

MQA Multi-Query Attention – Single KV head shared by all query heads; maximum cache reduction.

MLA Multi-head Latent Attention – DeepSeek's compressed KV projection via shared latent vectors.

RoPE Rotary Position Embedding – Encodes relative position via rotation matrices in attention.

iRoPE Interleaved RoPE – Alternates positional and non-positional attention layers for extreme context lengths.

BPE Byte-Pair Encoding – Subword tokenization algorithm merging frequent character pairs.

KV Cache Key-Value Cache – Stored attention states for efficient autoregressive generation.

FSDP Fully Sharded Data Parallelism – Distributes optimizer state, gradients, and parameters across devices.

PPO Proximal Policy Optimization – RL algorithm used in RLHF; requires a critic model.

CAI Constitutional AI – Self-critique alignment against a set of explicit constitutional principles.

RLAIF Reinforcement Learning from AI Feedback – AI-generated preferences replacing human annotations.

PRM Process Reward Model – Assigns reward scores to individual reasoning steps rather than final outputs.

CoT Chain-of-Thought – Prompting or training for step-by-step intermediate reasoning before answers.

MCTS Monte Carlo Tree Search – Tree-based search for the best reasoning path at test time.

TLT Token-Level Training – Draft-verify framework accelerating both training and inference.

TIES Trim, Elect, Sign – Model merging algorithm that handles conflicting weight updates.

DARE Drop And REscale – Model merging via random delta dropout with magnitude rescaling.

EWC Elastic Weight Consolidation – Prevents catastrophic forgetting via Fisher information penalty.

Appendix B

Recommended Reading

1. A. Vaswani et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. URL: https://arxiv.org/abs/1706.03762
2. L. Ouyang et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems*. Vol. 35. URL: https://arxiv.org/abs/2203.02155
3. R. Rafailov et al. (2023). "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems*. Vol. 36. URL: https://arxiv.org/abs/2305.18290
4. N. Shazeer et al. (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/1701.06538
5. D. Guo et al. (2025). "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning". In: *arXiv preprint arXiv:2501.12948*. URL: https://arxiv.org/abs/2501.12948
6. W. Fedus et al. (2022). "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity". In: *Journal of Machine Learning Research* 23.120, pp. 1–39. URL: https://arxiv.org/abs/2101.03961
7. E. J. Hu et al. (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2106.09685
8. Y. Bai et al. (2022). "Constitutional AI: Harmlessness from AI Feedback". In: *arXiv preprint arXiv:2212.08073*. URL: https://arxiv.org/abs/2212.08073
9. J. Hoffmann et al. (2022). "Training Compute-Optimal Large Language Models". In: *arXiv preprint arXiv:2203.15556*. URL: https://arxiv.org/abs/2203.15556
10. J. Wei et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 35. URL: https://arxiv.org/abs/2201.11903

11. G. Ilharco et al. (2023). "Editing Models with Task Arithmetic". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2212.04089
12. C. Snell et al. (2024). "Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters". In: *arXiv preprint arXiv:2408.03314*. URL: https://arxiv.org/abs/2408.03314
13. H. Touvron et al. (2023). "Llama 2: Open Foundation and Fine-Tuned Chat Models". In: *arXiv preprint arXiv:2307.09288*. URL: https://arxiv.org/abs/2307.09288
14. R. Anil et al. (2023). "Gemini: A Family of Highly Capable Multimodal Models". In: *arXiv preprint arXiv:2312.11805*. URL: https://arxiv.org/abs/2312.11805
15. T. Dao (2024). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2307.08691
16. C. Huyen (2025). *AI Engineering: Building Applications with Foundation Models*. ISBN 978-1-098-16685-5. O'Reilly Media
17. *NLP with Transformers* (2025 rev. ed.) – O'Reilly – Hugging Face authors

Appendix C

Hardware Reference

GPU / Accelerator	VRAM	FP16 TF	Best For
NVIDIA B200 SXM	192 GB HBM3e	2,250	Next-gen frontier training
NVIDIA GB200 NVL72	13.5 TB (rack)	130,000	Rack-scale cluster training
NVIDIA H200	141 GB HBM3e	989	Large models, long context
NVIDIA H100 SXM	80 GB HBM3	989	Frontier pre-training and RL
NVIDIA A100 80GB	80 GB HBM2e	312	General training/inference
NVIDIA L40S	48 GB GDDR6	362	Inference and fine-tuning
NVIDIA RTX 4090	24 GB GDDR6X	330	QLoRA, local inference
Google TPU v5e	16 GB HBM2e	197	Large-scale JAX/TF training
AMD MI300X	192 GB HBM3	1,307	Memory-intensive workloads

Table C.1: GPU and accelerator reference for LLM training (TFLOPS are peak BF16/FP16)

Appendix D

Quick-Start Recipes

D.1 QLoRA SFT with TRL and PEFT

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from transformers import AutoTokenizer
from peft import LoraConfig, get_peft_model
from trl import SFTConfig, SFTTrainer
from datasets import load_dataset

# Load model in 4-bit
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype="bfloat16",
)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B",
    quantization_config=bnb_config,
    device_map="auto",
)
tokenizer = AutoTokenizer.from_pretrained(
    "meta-llama/Llama-3.1-8B"
)

# LoRA config
lora_config = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, lora_config)

# Train
dataset = load_dataset("your_dataset")
```

```
trainer = SFTTrainer(
    model=model,
    train_dataset=dataset["train"],
    args=SFTConfig(
        output_dir="./output",
        num_train_epochs=3,
        per_device_train_batch_size=4,
        gradient_accumulation_steps=4,
        bf16=True,
    ),
)
trainer.train()
```

Listing D.1: Minimal QLoRA supervised fine-tuning with TRL

D.2 DPO Alignment with TRL

```
from trl import DPOConfig, DPOTrainer

# Dataset must have columns: prompt, chosen, rejected
dpo_trainer = DPOTrainer(
    model=model,            # SFT-trained model
    ref_model=ref_model,    # Frozen copy of the SFT model
    train_dataset=dataset,
    tokenizer=tokenizer,
    args=DPOConfig(
        beta=0.1,           # KL penalty coefficient
        max_length=1024,
        per_device_train_batch_size=2,
        gradient_accumulation_steps=8,
        bf16=True,
        output_dir="./dpo_output",
    ),
)
dpo_trainer.train()
```

Listing D.2: DPO preference alignment with TRL

D.3 GRPO Reasoning Training with TRL

```
from trl import GRPOConfig, GRPOTrainer

def reward_fn(completions, ground_truths, **kwargs):
    """Binary reward: 1.0 if answer matches, 0.0 otherwise."""
    rewards = []
    for completion, gt in zip(completions, ground_truths):
        answer = extract_boxed_answer(completion)
        rewards.append(1.0 if answer == gt else 0.0)
    return rewards
```

```
grpo_trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    train_dataset=math_dataset,
    args=GRPOConfig(
        num_generations=8,     # group size G
        max_new_tokens=512,
        temperature=0.9,
        beta=0.001,            # KL penalty weight
        output_dir="./grpo_output",
    ),
)
grpo_trainer.train()
```

Listing D.3: GRPO training for reasoning with verifiable rewards

D.4 Serving with vLLM

```
# CLI: vllm serve meta-llama/Llama-3.1-8B-Instruct \
#         --quantization awq \
#         --max-model-len 32768 \
#         --tensor-parallel-size 2 \
#         --enable-prefix-caching

from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Llama-3.1-8B-Instruct",
    quantization="awq",
    tensor_parallel_size=2,
    enable_prefix_caching=True,
)
sampling_params = SamplingParams(
    temperature=0.7, top_p=0.9, max_tokens=512
)
outputs = llm.generate(
    ["Explain transformer attention briefly."],
    sampling_params,
)
print(outputs[0].outputs[0].text)
```

Listing D.4: Launching a production vLLM OpenAI-compatible server

D.5 Model Merging with mergekit

```
# merge_config.yaml
models:
  - model: meta-llama/Llama-3.1-8B
```

```yaml
    # base model -- no parameters
  - model: ./checkpoints/llama3-coding-lora
    parameters:
      weight: 0.6
      density: 0.7       # DARE: keep 70% of delta params
  - model: ./checkpoints/llama3-math-lora
    parameters:
      weight: 0.4
      density: 0.7
merge_method: ties            # TIES-Merging
base_model: meta-llama/Llama-3.1-8B
parameters:
  normalize: true             # normalize final weights
dtype: bfloat16
```

Listing D.5: TIES-merging two LoRA fine-tunes with mergekit (YAML config)

```python
# Install: pip install mergekit
# Run merge:
# mergekit-yaml merge_config.yaml ./merged_model

# Quick evaluation after merge
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model = AutoModelForCausalLM.from_pretrained(
    "./merged_model",
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
tokenizer = AutoTokenizer.from_pretrained("./merged_model")

# Test merged capabilities
prompts = [
    "Write a Python function to compute fibonacci numbers.",
    "Solve: if 3x + 7 = 22, find x.",
]
for prompt in prompts:
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
    with torch.inference_mode():
        ids = model.generate(**inputs, max_new_tokens=200,
                             temperature=0.1, do_sample=True)
    print(tokenizer.decode(ids[0], skip_special_tokens=True))
    print("---")
```

Listing D.6: Running mergekit and evaluating the merged model

D.6 Continual Pre-training on Domain Data

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```python
from transformers import Trainer, TrainingArguments,
    DataCollatorForLanguageModeling
from datasets import load_dataset, concatenate_datasets

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B",
    torch_dtype="bfloat16",
    device_map="auto",
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B")

# Mix: 80% new domain data, 20% replay from original distribution
domain_data  = load_dataset("your_domain_corpus", split="train")
replay_data  = load_dataset("fineweb", split="train").select(range(50
    _000))
mixed_dataset = concatenate_datasets([domain_data, replay_data])
mixed_dataset = mixed_dataset.shuffle(seed=42)

def tokenize(examples):
    return tokenizer(examples["text"], truncation=True, max_length
    =2048)

tokenized = mixed_dataset.map(tokenize, batched=True,
                                remove_columns=["text"])
collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)

training_args = TrainingArguments(
    output_dir="./continual_pretrain",
    num_train_epochs=1,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=16,
    learning_rate=2e-5,           # lower LR for continual training
    warmup_steps=200,
    lr_scheduler_type="cosine",
    bf16=True,
    save_steps=500,
    logging_steps=50,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized,
    data_collator=collator,
)
trainer.train()
```

Listing D.7: Continual pre-training with replay to prevent forgetting

Appendix E

Cloud Provider Reference

E.1 Overview

GPU compute is available through major cloud providers, specialized AI cloud providers, and spot-market brokers. Pricing varies significantly by reservation type, region, and demand.

Provider	GPU Options	On-Demand Price	Best For
AWS	p4d.24xl (8×A100), p5.48xl (8×H100)	$32/hr – $98/hr	Enterprise, compliance
Google Cloud	A3 (8×H100), TPU v5p pods	$30/hr – $120/hr	JAX workloads, TPU
Azure	ND A100 v4, ND H100 v5	$32/hr – $100/hr	Microsoft ecosystem
Lambda Labs	8×H100 SXM	$24/hr	Cost-effective on-demand
CoreWeave	8×H100/A100	$20–$28/hr	Flexible allocation
RunPod	1–8× A100/H100	$2–$5/hr per GPU	Spot compute, research
Vast.ai	Community GPUs	$0.80–$3/hr	Budget fine-tuning

Table E.1: Cloud GPU pricing reference (April 2026; prices indicative, verify current rates)

E.2 Reservation vs. Spot Strategies

- **On-demand:** Full price; available immediately. Use for production inference or short experiments.
- **Reserved instances (1–3 years):** 30–60% discount vs. on-demand. Commit to this for sustained training workloads.
- **Spot / preemptible:** 60–90% cheaper; can be reclaimed with 30–120 seconds notice. Viable only with robust checkpoint-and-resume infrastructure. Run FSDP training with async checkpointing every 30 minutes.

- **Savings plans (AWS):** Commit to a spend level, not a specific instance type. Flexible across instance families.

E.3 Cost Estimation: Representative Workloads

Workload	Hardware	Duration	Est. Cost
QLoRA fine-tune 8B, 50K examples	1×A100 40GB	∼4 hours	$8–$12
Full fine-tune 8B, 100K examples	4×A100 80GB	∼8 hours	$120–$160
DPO alignment 8B	2×A100 80GB	∼6 hours	$60–$90
GRPO reasoning 7B	8×A100 80GB	∼24 hours	$600–$800
Pre-train 7B, 1T tokens	64×H100	∼21 days	$90K–$130K
Pre-train 70B, 2T tokens	512×H100	∼30 days	$1.5M–$2.5M

Table E.2: Cost estimates for representative LLM workloads (indicative, 2026)

E.4 Infrastructure Checklist

Before Starting a Long Training Run

- ✓Async checkpointing configured and tested (simulate a restart).
- ✓Monitoring dashboard: GPU utilization, loss curve, gradient norm, memory.
- ✓Evaluation harness running on a held-out set every N steps.
- ✓Alert on loss spikes $> 2\times$ rolling mean and OOM errors.
- ✓Estimated total cost approved; spot interruption rate acceptable.
- ✓Data pipeline throughput tested: can your dataloader saturate all GPUs?

Appendix F

Training Hyperparameter Reference

A consolidated reference for common LLM training scenarios.

F.1 Pre-training

Hyperparameter	7B Model	70B Model	Notes
Optimizer	AdamW	AdamW	Muon competitive at 7B
Learning rate (peak)	3×10^{-4}	1×10^{-4}	Scale down with model size
LR scheduler	Cosine + warmup	Cosine + warmup	WSD alternative
Warmup steps	2,000	4,000	More warmup for larger models
β_1	0.9	0.9	Standard
β_2	0.95	0.95	Slightly lower than Adam default
Weight decay	0.1	0.1	Higher than traditional 0.01
Gradient clip norm	1.0	1.0	Standard
Batch size (tokens)	4M	8M	Scale with model size
Sequence length	4,096	4,096	Extend in Stage 2
Precision	BF16	BF16 / FP8	FP8 for H100/H200

Table F.1: Pre-training hyperparameter reference

F.2 Supervised Fine-Tuning (SFT)

Hyperparameter	Typical Range	Notes
Learning rate	1×10^{-5} – 5×10^{-5}	Lower than pre-training; avoid forgetting
Epochs	2–5	3 is the most common sweet spot
Batch size	32–128	Larger = more stable; use grad accum
Scheduler	Cosine or linear decay	Linear often sufficient for short runs
Warmup ratio	0.03–0.1	3–10% of total steps
Max sequence length	2,048–8,192	Pack shorter sequences
Loss masking	On (assistant only)	Critical: never train on user/system tokens

Table F.2: SFT hyperparameter reference

F.3 LoRA / QLoRA

Hyperparameter	Typical Value	Notes
Rank r	8 – 64	16 is the most common default; higher r = more params
Alpha α	$2r$ (e.g., 32 for r=16)	α/r is the effective LR scale (usually 2)
Dropout	0.05 – 0.1	Low dropout for small datasets
Target modules	q_proj, v_proj	Add k_proj, o_proj for harder tasks
Learning rate	1×10^{-4} – 3×10^{-4}	Higher than full FT; adapter weights start near zero
QLoRA quantization	nf4 (4-bit NormalFloat)	nf4 outperforms fp4 for most LLMs
Compute dtype	BF16	Always use BF16 for LoRA computation

Table F.3: LoRA and QLoRA hyperparameter reference

F.4 DPO / SimPO Alignment

Hyperparameter	Typical Value	Notes
β (KL penalty)	0.01 – 0.5	Lower β = more aggressive policy update
Learning rate	5×10^{-7} – 1×10^{-5}	Much lower than SFT
Epochs	1 – 3	Over-training causes reward hacking
Batch size	8 – 64	Larger = more stable preference gradient
Max length	1,024 – 4,096	Truncate long responses symmetrically
Chosen/rejected ratio	1:1	Balanced preference pairs
SimPO γ margin	0.5 – 2.0	Target reward margin between chosen and rejected

Table F.4: DPO and SimPO hyperparameter reference

F.5 GRPO Reasoning Training

Hyperparameter	Typical Value	Notes
Group size G	8 – 16	DeepSeek uses 16; larger = more stable advantage estimate
β (KL weight)	0.001 – 0.01	Very small; reward signal dominates
Temperature (sampling)	0.7 – 1.0	Higher = more diverse generations for better advantage estimation
Max new tokens	512 – 4096	Match the complexity of reasoning tasks
Learning rate	1×10^{-6} – 5×10^{-6}	Lower than SFT; RL updates are noisier
Reward normalization	Group mean/std	Normalize within the group, not globally
Clip ratio ε	0.2	Standard PPO-style clipping

Table F.5: GRPO hyperparameter reference

F.6 Common Failure Modes and Fixes

Symptom	Likely Cause	Fix
Loss NaN early in training	LR too high or warmup too short	Halve LR; double warmup steps
Loss plateau without improvement	LR too low or dataset too small	Increase LR by 2×; check data quality
Loss spike then recovery	Corrupted data batch	Add data validation; inspect spike step's batch
Gradient norm spikes	Outlier data or no clipping	Enable grad clip norm 1.0; filter data
Model repeats outputs	Training data contamination or over-training	Reduce epochs; check for dedup failures
DPO loss goes negative	β too small or LR too high	Increase β; reduce LR by 5×
GRPO reward stuck at zero	Reward function too strict or group too small	Soften reward; increase group size
OOM during GRPO	Too many generations in memory	Reduce group size; use gradient checkpointing

Table F.6: Common training failure modes and remediation

Appendix G

The Implementation Treasury: Theory & Code

This appendix provides a unified reference for the mathematical formulations and PyTorch implementations discussed throughout the book. Each theoretical concept is paired with a self-contained code snippet for immediate practical application.

G.1 Shared Foundation

All code snippets in this treasury assume the following standard imports and PyTorch environment:

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import Optional, List
```

Listing G.1: Shared imports

G.2 Optimization Objectives

A production LLM is produced by five sequential optimisation stages.

G.2.1 Stage 1 — Pre-training

Objective (causal language modelling):

$$\theta_1^* = \arg\min_{\theta} \mathcal{L}_{\mathrm{PT}}(\theta) = -\frac{1}{|\mathcal{D}_{\mathrm{PT}}|} \sum_{(x_1,\ldots,x_T)\in\mathcal{D}_{\mathrm{PT}}} \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

```
def causal_lm_loss(logits: torch.Tensor, labels: torch.Tensor) ->
    torch.Tensor:
     """Computes cross-entropy loss by shifting logits and labels for
    NTP.
     Labels with -100 are ignored (standard for SFT masking).
     """
     # Shift: Predict token at t from tokens 0 to t-1
     shift_logits = logits[:, :-1, :].contiguous().view(-1, logits.
    size(-1))
     shift_labels = labels[:, 1:].contiguous().view(-1)
     return F.cross_entropy(shift_logits, shift_labels, ignore_index
    =-100)
```

Listing G.2: Causal LM loss with token masking

G.2.2 Perplexity (PPL)

$$\mathrm{PPL} = \exp(\mathcal{L}_{\mathrm{NTP}})$$

Perplexity measures the model's confidence in its predictions; it represents the average branching factor of the distribution.

G.2.3 Stage 3 — Preference Optimisation

DPO (Direct Preference Optimization):

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)}\right)\right]$$

SimPO (Simple Preference Optimization):

$$\mathcal{L}_{\mathrm{SimPO}} = -\mathbb{E}\left[\log\sigma\left(\frac{\beta}{|y_w|}\log\pi_\theta(y_w|x) - \frac{\beta}{|y_l|}\log\pi_\theta(y_l|x) - \gamma\right)\right]$$

GRPO (Group Relative Policy Optimisation):

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r}$$

G.3 Attention Mechanisms

G.3.1 Scaled Dot-Product Attention

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

```python
def scaled_dot_product_attention(
    Q: torch.Tensor,    # (Batch, Heads, QueryLen, Dim)
    K: torch.Tensor,    # (Batch, Heads, KeyLen, Dim)
    V: torch.Tensor,    # (Batch, Heads, KeyLen, ValDim)
    causal: bool = False,
) -> torch.Tensor:
    """Computes scaled dot-product attention with optional causal
   masking."""
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)

    if causal:
        T, S = scores.shape[-2], scores.shape[-1]
        mask = torch.triu(torch.ones(T, S, device=Q.device), diagonal
   =1).bool()
        scores = scores.masked_fill(mask, float('-inf'))

    attn_weights = F.softmax(scores, dim=-1)
    return attn_weights @ V
```

Listing G.3: Scaled dot-product attention

G.3.2 Multi-Head Attention (MHA)

$$\text{head}_h = \text{Attention}(QW_h^Q, KW_h^K, VW_h^V), \quad \text{MHA}(Q,K,V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)W^O$$

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        self.h, self.d_k = n_heads, d_model // n_heads
        self.Wq = nn.Linear(d_model, d_model, bias=False)
        self.Wk = nn.Linear(d_model, d_model, bias=False)
        self.Wv = nn.Linear(d_model, d_model, bias=False)
        self.Wo = nn.Linear(d_model, d_model, bias=False)

    def _split_into_heads(self, x: torch.Tensor) -> torch.Tensor:
        B, T, _ = x.shape
        return x.view(B, T, self.h, self.d_k).transpose(1, 2)

    def forward(self, x: torch.Tensor, causal: bool = True) -> torch.
   Tensor:
        Q = self._split_into_heads(self.Wq(x))
        K = self._split_into_heads(self.Wk(x))
        V = self._split_into_heads(self.Wv(x))
        out = scaled_dot_product_attention(Q, K, V, causal=causal)
        B, H, T, dk = out.shape
        out = out.transpose(1, 2).reshape(B, T, H * dk)
        return self.Wo(out)
```

Listing G.4: Multi-head attention

G.3.3 Grouped Query Attention (GQA)

KV groups shared by H/G query heads. $G = 1$ is MQA; $G = H$ is MHA.

```
class GroupedQueryAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int, n_kv_groups: int):
        super().__init__()
        self.h, self.g = n_heads, n_kv_groups
        self.d_k = d_model // n_heads
        self.Wq = nn.Linear(d_model, n_heads * self.d_k, bias=False)
        self.Wk = nn.Linear(d_model, n_kv_groups * self.d_k, bias=
   False)
        self.Wv = nn.Linear(d_model, n_kv_groups * self.d_k, bias=
   False)
        self.Wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x: torch.Tensor, causal: bool = True) -> torch.
   Tensor:
        B, T, _ = x.shape
        q = self.Wq(x).view(B, T, self.h, self.d_k).transpose(1, 2)
        k = self.Wk(x).view(B, T, self.g, self.d_k).transpose(1, 2)
        v = self.Wv(x).view(B, T, self.g, self.d_k).transpose(1, 2)
        reps = self.h // self.g
        k = k.repeat_interleave(reps, dim=1)
        v = v.repeat_interleave(reps, dim=1)
        out = scaled_dot_product_attention(q, k, v, causal=causal)
        out = out.transpose(1, 2).reshape(B, T, self.h * self.d_k)
        return self.Wo(out)
```

Listing G.5: Grouped query attention

G.3.4 Multi-Query Attention (MQA)

Shares a single KV pair across all query heads. Special case of GQA where $G = 1$.

```
class MultiQueryAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        self.h, self.d_k = n_heads, d_model // n_heads
        self.Wq = nn.Linear(d_model, d_model, bias=False)
        self.Wk = nn.Linear(d_model, self.d_k, bias=False)
        self.Wv = nn.Linear(d_model, self.d_k, bias=False)
        self.Wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x: torch.Tensor, causal: bool = True) -> torch.
   Tensor:
        B, T, _ = x.shape
        q = self.Wq(x).view(B, T, self.h, self.d_k).transpose(1, 2)
        k = self.Wk(x).view(B, T, 1, self.d_k).transpose(1, 2)
        v = self.Wv(x).view(B, T, 1, self.d_k).transpose(1, 2)
        out = scaled_dot_product_attention(q, k, v, causal=causal)
        out = out.transpose(1, 2).reshape(B, T, self.h * self.d_k)
```

```
        return self.Wo(out)
```

Listing G.6: Multi-query attention

G.3.5 Multi-head Latent Attention (MLA)

$$c_t^{KV} = W^{DKV} h_t, \quad [k_t^C; v_t^C] = W^{UKV} c_t^{KV}$$

```
class MLA(nn.Module):
    def __init__(self, d_model: int, n_heads: int, d_latent: int,
   d_v_latent: int):
        super().__init__()
        self.h, self.d_k = n_heads, d_model // n_heads
        self.W_dk_c = nn.Linear(d_model, d_latent, bias=False)
        self.W_uk = nn.Linear(d_latent, n_heads * self.d_k, bias=
   False)
        self.W_uv = nn.Linear(d_latent, n_heads * self.d_k, bias=
   False)
        self.Wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x: torch.Tensor, causal: bool = True) -> torch.
   Tensor:
        B, T, _ = x.shape
        c_kv = self.W_dk_c(x)
        k = self.W_uk(c_kv).view(B, T, self.h, self.d_k).transpose(1,
    2)
        v = self.W_uv(c_kv).view(B, T, self.h, self.d_k).transpose(1,
    2)
        q = nn.Linear(x.shape[-1], self.h * self.d_k, device=x.device
   )(x).view(B, T, self.h, self.d_k).transpose(1, 2)
        out = scaled_dot_product_attention(q, k, v, causal=causal)
        out = out.transpose(1, 2).reshape(B, T, self.h * self.d_k)
        return self.Wo(out)
```

Listing G.7: Multi-head latent attention (MLA)

G.4 Positional Encodings

G.4.1 Rotary Position Embedding (RoPE)

$$R_{\theta_i,t} = \begin{pmatrix} \cos(t\theta_i) & -\sin(t\theta_i) \\ \sin(t\theta_i) & \cos(t\theta_i) \end{pmatrix}, \quad \theta_i = 10000^{-2(i-1)/d}$$

```
def apply_rotary_emb(x: torch.Tensor, freqs: torch.Tensor) -> torch.
   Tensor:
    """Applies complex rotary embeddings to the last dimension of x."
   """
    x_complex = torch.view_as_complex(x.float().reshape(*x.shape
   [:-1], -1, 2))
```

```
    freqs = freqs.view(1, 1, x.shape[-2], -1)
    x_rotated = x_complex * torch.exp(1j * freqs)
    return torch.view_as_real(x_rotated).flatten(-2).type_as(x)

def precompute_freqs(dim: int, seq_len: int, theta: float = 10000.0)
   -> torch.Tensor:
    """Precomputes the rotary frequency tensor for a given context
   length."""
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].
   float() / dim))
    t = torch.arange(seq_len)
    return torch.outer(t, freqs).float()
```

Listing G.8: Rotary positional embeddings (RoPE)

G.4.2 ALiBi (Attention with Linear Biases)

$$\text{score}_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} - m_h \cdot |i - j|$$

```
def alibi_bias(n_heads: int, seq_len: int) -> torch.Tensor:
    """Generates the linear penalty matrix for ALiBi attention."""
    slopes = 2 ** (-8 * torch.arange(1, n_heads + 1) / n_heads)
    queries = torch.arange(seq_len).unsqueeze(0)
    keys    = torch.arange(seq_len).unsqueeze(1)
    pos     = queries - keys
    bias = -slopes.view(-1, 1, 1) * pos.abs().unsqueeze(0).float()
    return bias.tril()
```

Listing G.9: ALiBi bias matrix

G.5 Normalization and Activation

G.5.1 RMSNorm

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\|x\|_{\text{RMS}} + \epsilon}, \quad \|x\|_{\text{RMS}} = \sqrt{\frac{1}{d}\sum_i x_i^2}$$

```
class RMSNorm(nn.Module):
    """Root Mean Square Layer Normalization (standard in Llama 2/3)."
   ""
    def __init__(self, d_model: int, eps: float = 1e-8):
        super().__init__()
        self.eps, self.gamma = eps, nn.Parameter(torch.ones(d_model))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        rms = x.pow(2).mean(-1, keepdim=True).add(self.eps).sqrt()
        return self.gamma * x / rms
```

Listing G.10: RMSNorm

G.5.2 SwiGLU Feed-Forward Network

$$\mathrm{SwiGLU}(x, W, V, W_2) = (xW \odot \mathrm{Swish}(xV))W_2$$

```
class SwiGLU(nn.Module):
    """Gated Linear Unit with Swish activation (standard in LLM FFNs)
    """
    def __init__(self, d_model: int, d_ff: int):
        super().__init__()
        self.W1 = nn.Linear(d_model, d_ff, bias=False)
        self.W2 = nn.Linear(d_model, d_ff, bias=False)
        self.W3 = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.W3(F.silu(self.W1(x)) * self.W2(x))
```

Listing G.11: SwiGLU FFN

G.6 Optimization

G.6.1 AdamW

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right)$$

```
def adamw_step(
    params: list[torch.Tensor], grads: list[torch.Tensor],
    m: list[torch.Tensor], v: list[torch.Tensor], t: int,
    lr: float = 3e-4, b1: float = 0.9, b2: float = 0.95,
    eps: float = 1e-8, wd: float = 0.1
) -> int:
    """Manual implementation of the AdamW update rule with decoupled
    weight decay."""
    t += 1
    for p, g, m_i, v_i in zip(params, grads, m, v):
        m_i.mul_(b1).add_(g, alpha=1 - b1)
        v_i.mul_(b2).addcmul_(g, g, value=1 - b2)
        m_hat = m_i / (1 - b1 ** t)
        v_hat = v_i / (1 - b2 ** t)
        p.mul_(1 - lr * wd)
        p.addcdiv_(m_hat, v_hat.sqrt().add_(eps), value=-lr)
    return t
```

Listing G.12: AdamW update step (manual)

G.6.2 Muon Optimizer

$$X_{new} = 0.5X(3I - XX^\top)$$

```
@torch.no_grad()
def muon_step(p, g, m, lr=0.02, momentum=0.95, n_iters=5):
    """Orthogonalizing optimizer for the 2026 frontier."""
    m.mul_(momentum).add_(g)
    g = g.add(m, alpha=momentum)
    X = g.view(g.size(0), -1)
    X /= X.norm().clamp(min=1e-7)
    for _ in range(n_iters):
        X = 0.5 * X @ (3.0 * torch.eye(X.size(0), device=X.device) -
   X @ X.T)
    p.add_(X.view_as(p), alpha=-lr)
```

Listing G.13: Muon update step

G.6.3 Lion Optimizer

$$c_t = \text{sign}(\beta_1 m_{t-1} + (1-\beta_1)g_t), \quad m_t = \beta_2 m_{t-1} + (1-\beta_2)g_t$$

Lion uses the ‘sign‘ operation to determine update direction, providing better generalization in specific LLM regimes.

G.6.4 Cosine Schedule with Warmup

```
def cosine_lr(step: int, total_steps: int, lr_max: float, lr_min:
   float, warmup_steps: int) -> float:
    if step < warmup_steps:
        return lr_max * step / warmup_steps
    progress = (step - warmup_steps) / max(1, total_steps -
   warmup_steps)
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + math.cos(math.pi *
    progress))
```

Listing G.14: Cosine LR schedule with warmup

G.7 Parameter-Efficient Fine-Tuning

G.7.1 LoRA (Low-Rank Adaptation)

$$h = W_0 x + \frac{\alpha}{r} BAx$$

```
class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, rank: int = 16, alpha: float
   = 32.0):
        super().__init__()
        d_out, d_in = base.weight.shape
        self.base, self.scale = base, alpha / rank
        self.base.weight.requires_grad_(False)
        self.A = nn.Parameter(torch.randn(rank, d_in) * 0.01)
        self.B = nn.Parameter(torch.zeros(d_out, rank))
```

```
    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale
```

Listing G.15: LoRA linear layer

G.7.2 DoRA (Weight-Decomposed Low-Rank Adaptation)

$$W = m\frac{W_0 + BA}{\|W_0 + BA\|_F}$$

G.7.3 PPO (Proximal Policy Optimization)

$$\mathcal{L}_{\text{CLIP}} = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$$

```
def ppo_loss(policy_logits, old_logits, actions, advantages, eps=0.2)
    :
    ratio = (policy_logits - old_logits).exp()
    surr1 = ratio * advantages
    surr2 = torch.clamp(ratio, 1-eps, 1+eps) * advantages
    return -torch.min(surr1, surr2).mean()
```

Listing G.16: PPO clipped objective

G.8 Alignment Objectives

G.8.1 DPO (Direct Preference Optimization)

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

```
def dpo_loss(policy, ref, x_w, x_l, beta=0.1):
    """Computes the DPO loss for a pair of responses."""
    pi_w, pi_l = log_prob_seq(policy, x_w, x_w), log_prob_seq(policy,
    x_l, x_l)
    ref_w, ref_l = log_prob_seq(ref, x_w, x_w), log_prob_seq(ref, x_l
    , x_l)
    reward_margin = beta * ((pi_w - ref_w) - (pi_l - ref_l))
    return -F.logsigmoid(reward_margin).mean()
```

Listing G.17: Direct Preference Optimization loss

G.8.2 SimPO (Simple Preference Optimization)

$$\mathcal{L}_{\text{SimPO}} = -\mathbb{E}\left[\log\sigma\left(\frac{\beta}{|y_w|}\log\pi_\theta(y_w|x) - \frac{\beta}{|y_l|}\log\pi_\theta(y_l|x) - \gamma\right)\right]$$

```
def simpo_loss(model, x_w, x_l, beta=2.0, gamma=0.5):
    """Length-normalized preference optimization without a reference
   model."""
    r_w, r_l = avg_log_prob(model, x_w, x_w), avg_log_prob(model, x_l
   , x_l)
    return -F.logsigmoid(beta * (r_w - r_l) - gamma).mean()
```

Listing G.18: SimPO loss

G.8.3 GRPO (Group Relative Policy Optimisation)

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r}, \quad \mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}\left[\sum_{i=1}^{G}\sum_{t}\cdots\right]$$

```
def grpo_step(
    policy: nn.Module,
    ref: nn.Module,
    prompt_ids: torch.Tensor,
    reward_fn: callable,
    G: int = 8,
    beta: float = 0.001,
    eps: float = 0.2
) -> torch.Tensor:
    """Group Relative Policy Optimization update step."""
    with torch.no_grad():
        completions = [policy.generate(prompt_ids) for _ in range(G)]
        rewards     = reward_fn(completions)
        old_logp    = torch.stack([log_prob_seq(policy, c, c) for c
   in completions])

    advantage = (rewards - rewards.mean()) / (rewards.std() + 1e-8)

    total_batch_loss = 0.0
    for i, comp in enumerate(completions):
        curr_logp = log_prob_seq(policy, comp, comp)
        ratio     = (curr_logp - old_logp[i]).exp()

        surr1     = ratio * advantage[i]
        surr2     = torch.clamp(ratio, 1 - eps, 1 + eps) * advantage[
   i]
        pg_loss   = -torch.min(surr1, surr2)

        kl_div    = curr_logp - log_prob_seq(ref, comp, comp)
        total_batch_loss += pg_loss + beta * kl_div

    return total_batch_loss / G
```

Listing G.19: GRPO update step

G.8.4 KTO (Kahneman-Tversky Optimization)

$$\mathcal{L}_{\text{KTO}} = -\mathbb{E}[w \cdot \log \sigma(\beta(r(x, y) - \text{ref}))]$$

```
def kto_loss(model, ref, input_ids, labels, beta=0.1):
    """Alignment from binary feedback using Kahneman-Tversky utility.
   """
    logp = log_prob_seq(model, input_ids, labels)
    logp_ref = log_prob_seq(ref, input_ids, labels)
    return -F.logsigmoid(beta * (logp - logp_ref)).mean()
```

Listing G.20: KTO loss

G.9 Inference and Distillation

G.9.1 Knowledge Distillation (Forward KL)

$$\mathcal{L}_{\text{fwd}} = D_{\text{KL}}(p_{\text{teacher}} \| p_{\text{student}})$$

```
def fwd_kl_distill(s_logits, t_logits, temp=2.0):
    p = F.softmax(t_logits / temp, dim=-1)
    log_q = F.log_softmax(s_logits / temp, dim=-1)
    return F.kl_div(log_q, p, reduction="batchmean") * (temp**2)
```

Listing G.21: Forward KL distillation

G.9.2 MiniLLM (Reverse KL)

$$\mathcal{L}_{\text{rev}} = D_{\text{KL}}(p_{\text{student}} \| p_{\text{teacher}})$$

G.9.3 KV Cache Management

The memory footprint (in bytes) of the KV cache is:

$$M_{KV} = 2 \times L \times T \times H \times d_k \times \text{bytes_per_param}$$

where L is layers, T is sequence length, and H is heads.

G.9.4 Quantization Scaling

Linear quantization to n bits (e.g., INT8):

$$x_q = \text{round}\left(\frac{x}{\Delta}\right), \quad \Delta = \frac{\max(|x|)}{2^{n-1} - 1}$$

G.9.5 Speculative Decoding

Accept token x_t with probability $\alpha(x_t) = \min(1, p(x_t)/q(x_t))$.

```
@torch.no_grad()
def speculative_decode(
    target: nn.Module,
    draft: nn.Module,
    input_ids: torch.Tensor,
    max_new: int = 128,
    k: int = 4
) -> torch.Tensor:
    """Accelerated inference via draft-target asymmetric sampling."""
    ids = input_ids.clone()
    while ids.shape[1] < input_ids.shape[1] + max_new:
        draft_ids, draft_probs = ids.clone(), []
        for _ in range(k):
            d_logits = draft(draft_ids).logits[:, -1]
            p_draft  = F.softmax(d_logits, dim=-1)
            tok      = torch.multinomial(p_draft, 1)
            draft_probs.append(p_draft[0, tok[0, 0]].item())
            draft_ids = torch.cat([draft_ids, tok], dim=1)

        t_logits = target(draft_ids).logits
        accepted = 0
        for j in range(k):
            p_tgt   = F.softmax(t_logits[0, ids.shape[1]+j-1], dim
   =-1)
            tok_j   = draft_ids[0, ids.shape[1] + j].item()
            alpha   = min(1.0, p_tgt[tok_j].item() / (draft_probs[j]
   + 1e-9))
            if torch.rand(1).item() < alpha:
                accepted += 1
            else:
                break

        ids = torch.cat([ids, draft_ids[:, ids.shape[1]:ids.shape[1]+
   accepted+1]], dim=1)
    return ids
```

Listing G.22: Speculative decoding loop

G.10 Model Merging

G.10.1 Task Vectors

$$\tau = \theta_{fine-tuned} - \theta_{base}$$

G.10.2 SLERP (Spherical Linear Interpolation)

$$\mathrm{SLERP}(\theta_A, \theta_B, t) = \frac{\sin((1-t)\Omega)}{\sin\Omega}\theta_A + \frac{\sin(t\Omega)}{\sin\Omega}\theta_B$$

```
def slerp(w_a: torch.Tensor, w_b: torch.Tensor, t: float) -> torch.
    Tensor:
    """Interpolates between weights along an arc for better
    distribution preservation."""
    a, b = w_a.flat().float(), w_b.flat().float()
    cos_omega = (a / a.norm()).dot(b / b.norm()).clamp(-1, 1)
    omega = cos_omega.acos()
    if omega.abs() < 1e-6: return (1-t)*w_a + t*w_b
    return (math.sin((1-t)*omega)*a + math.sin(t*omega)*b) / math.sin
    (omega)
```

Listing G.23: SLERP weight interpolation

G.10.3 DARE (Drop And REscale)

$$\theta_{merge} = \theta_{base} + \frac{1}{1-p}\sum_{i=1}^{n} \text{mask}_i \odot \tau_i$$

where p is the dropout probability.

G.10.4 TIES-Merging

Three-step conflict resolution: Trim, Elect sign, and Disjoint merge.

```
def task_vector(base_sd: dict, ft_sd: dict) -> dict:
    """Computes the delta vector: theta_diff = theta_ft - theta_base.
    """
    return {k: ft_sd[k].float() - base_sd[k].float() for k in base_sd
    }
```

Listing G.24: Task Vector

```
def ties_merge(base_sd: dict, ft_sds: list[dict], lambdas: list[float
    ], density: float = 0.7) -> dict:
    """Trim, Elect, and Merge conflict resolution."""
    taus = [task_vector(base_sd, ft) for ft in ft_sds]
    trimmed = []
    # ... (Trimming and logic)
    return merged
```

Listing G.25: TIES-Merging

G.11 Continual Learning — EWC

G.11.1 Elastic Weight Consolidation (EWC)

```
class EWC:
    def __init__(self, model, dataset, lam=1000.0):
        self.lam, self.params = lam, {n: p.clone().detach() for n, p
    in model.named_parameters()}
```

```
        self.fisher = self._compute_fisher(model, dataset)
    # ... (Penalty and Fisher logic)
```

Listing G.26: Elastic Weight Consolidation

G.12 Final Assembly — TinyGPT

A minimal, production-ready decoder architecture incorporating the treasury's components.

```
class DecoderLayer(nn.Module):
    def __init__(self, d_model: int = 512, n_heads: int = 8, n_kv:
   int = 2, d_ff: int = 2048):
        super().__init__()
        self.norm1, self.norm2 = RMSNorm(d_model), RMSNorm(d_model)
        self.attn, self.ffn = GroupedQueryAttention(d_model, n_heads,
    n_kv), SwiGLU(d_model, d_ff)

    def forward(self, x: torch.Tensor, freqs: torch.Tensor) -> torch.
   Tensor:
        x = x + self.attn(self.norm1(x))
        return x + self.ffn(self.norm2(x))

class TinyGPT(nn.Module):
    def __init__(self, vocab=32000, d=512, heads=8, kv=2, ff=2048,
   layers=6, max_seq=2048):
        super().__init__()
        self.embed   = nn.Embedding(vocab, d)
        self.layers  = nn.ModuleList([DecoderLayer(d, heads, kv, ff)
   for _ in range(layers)])
        self.norm, self.head = RMSNorm(d), nn.Linear(d, vocab, bias=
   False)
        self.head.weight = self.embed.weight
        self.register_buffer("freqs", precompute_freqs(d // heads,
   max_seq))

    def forward(self, idx: torch.Tensor) -> torch.Tensor:
        x = self.embed(idx)
        for layer in self.layers: x = layer(x, self.freqs)
        return self.head(self.norm(x))
```

Listing G.27: Complete TinyGPT architecture

References

Agarwal, R., N. Vieillot, M. G. Azar, P. Stanczyk, D. Calandriello, R. Munos, and B. Piot (2024). "On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes". In: *arXiv preprint arXiv:2306.13649*. URL: https://arxiv.org/abs/2306.13649 (cit. on p. 43).

Ainslie, J., J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. URL: https://arxiv.org/abs/2305.13245 (cit. on p. 6).

Anil, R., S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, et al. (2023). "Gemini: A Family of Highly Capable Multimodal Models". In: *arXiv preprint arXiv:2312.11805*. URL: https://arxiv.org/abs/2312.11805 (cit. on p. 72).

Azar, M. G., Z. D. Guo, B. Piot, R. Munos, M. Rowland, M. Valko, and D. Calandriello (2024). "A General Theoretical Paradigm to Understand Learning from Human Feedback". In: *arXiv preprint arXiv:2310.12036*. URL: https://arxiv.org/abs/2310.12036 (cit. on p. 34).

Bai, Y., S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. (2022). "Constitutional AI: Harmlessness from AI Feedback". In: *arXiv preprint arXiv:2212.08073*. URL: https://arxiv.org/abs/2212.08073 (cit. on pp. 35, 71).

Brown, T. B., B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. (2020). "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33. URL: https://arxiv.org/abs/2005.14165 (cit. on p. 4).

Chen, X., C. Liang, D. Huang, E. Real, K. Wang, Y. Liu, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, et al. (2023). "Symbolic Discovery of Optimization Algorithms". In: *Advances in Neural Information Processing Systems*. Vol. 36. URL: https://arxiv.org/abs/2302.06675 (cit. on p. 27).

Dai, D., C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, et al. (2024). "DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models". In: *arXiv preprint arXiv:2401.06066*. URL: https://arxiv.org/abs/2401.06066 (cit. on p. 7).

Dao, T. (2024). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2307.08691 (cit. on pp. 8, 72).

Dao, T., D. Y. Fu, S. Ermon, A. Rudra, and C. Ré (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems*. Vol. 35. URL: https://arxiv.org/abs/2205.14135 (cit. on p. 8).

Dettmers, T., M. Lewis, Y. Belkada, and L. Zettlemoyer (2022). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *Advances in Neural Information Processing Systems*. Vol. 35. URL: https://arxiv.org/abs/2208.07339 (cit. on p. 52).

Dettmers, T., A. Pagnoni, A. Holtzman, and L. Zettlemoyer (2023). "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems*. Vol. 36. URL: https://arxiv.org/abs/2305.14314 (cit. on p. 32).

Ethayarajh, K., W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela (2024). "KTO: Model Alignment as Prospect Theoretic Optimization". In: *arXiv preprint arXiv:2402.01306*. URL: https://arxiv.org/abs/2402.01306 (cit. on p. 34).

Fedus, W., B. Zoph, and N. Shazeer (2022). "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity". In: *Journal of Machine Learning Research* 23.120, pp. 1–39. URL: https://arxiv.org/abs/2101.03961 (cit. on p. 71).

Gu, Y., L. Dong, F. Wei, and M. Huang (2024). "MiniLLM: Knowledge Distillation of Large Language Models". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2306.08543 (cit. on p. 42).

Guo, D., D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al. (2025). "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning". In: *arXiv preprint arXiv:2501.12948*. URL: https://arxiv.org/abs/2501.12948 (cit. on pp. 34, 39, 66, 71).

Hoffmann, J., S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. (2022). "Training Compute-Optimal Large Language Models". In: *arXiv preprint arXiv:2203.15556*. URL: https://arxiv.org/abs/2203.15556 (cit. on pp. 4, 71).

Hu, E. J., Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2106.09685 (cit. on pp. 32, 71).

Huyen, C. (2025). *AI Engineering: Building Applications with Foundation Models*. ISBN 978-1-098-16685-5. O'Reilly Media (cit. on p. 72).

Ilharco, G., M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi (2023). "Editing Models with Task Arithmetic". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2212.04089 (cit. on pp. 60, 72).

Kim, Y. and A. M. Rush (2016). "Sequence-Level Knowledge Distillation". In: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. URL: https://arxiv.org/abs/1606.07947 (cit. on p. 42).

Kirkpatrick, J., R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. (2017). "Overcoming Catastrophic Forgetting in Neural Networks". In: *Proceedings of the National Academy of Sciences* 114.13, pp. 3521–3526. URL: https://arxiv.org/abs/1612.00796 (cit. on p. 61).

Kosson, A., B. Messmer, and M. Jaggi (2024). "Modular Adaptive Optimization for Large Language Models". In: *arXiv preprint arXiv:2409.20325*. URL: https://arxiv.org/abs/2409.20325 (cit. on p. 27).

Kudo, T. and J. Richardson (2018). "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing". In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. URL: https://arxiv.org/abs/1808.06226 (cit. on p. 15).

Kwon, W., Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*. URL: https://arxiv.org/abs/2309.06180 (cit. on p. 52).

Leviathan, Y., M. Kalman, and Y. Matias (2023). "Fast Inference from Transformers via Speculative Decoding". In: *International Conference on Machine Learning*. URL: https://arxiv.org/abs/2211.17192 (cit. on pp. 38, 43, 52).

Lightman, H., V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe (2023). "Let's Verify Step by Step". In: *arXiv preprint arXiv:2305.20050*. URL: https://arxiv.org/abs/2305.20050 (cit. on pp. 34, 38).

Liu, H., Z. Li, D. Hall, P. Liang, and T. Ma (2023). "Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training". In: *arXiv preprint arXiv:2305.14342*. URL: https://arxiv.org/abs/2305.14342 (cit. on p. 27).

Liu, S.-Y., C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen (2024). "DoRA: Weight-Decomposed Low-Rank Adaptation". In: *International Conference on Machine Learning*. URL: https://arxiv.org/abs/2402.09353 (cit. on p. 32).

Loshchilov, I. and F. Hutter (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/1711.05101 (cit. on p. 26).

Meng, Y., M. Xia, and D. Chen (2024). "SimPO: Simple Preference Optimization with a Reference-Free Reward". In: *arXiv preprint arXiv:2405.14734*. URL: https://arxiv.org/abs/2405.14734 (cit. on p. 34).

Mukherjee, S., A. Mitra, G. Jawahar, S. Agarwal, H. Palangi, and A. Awadallah (2023). "Orca: Progressive Learning from Complex Explanation Traces of GPT-4". In: *arXiv preprint arXiv:2306.02707*. URL: https://arxiv.org/abs/2306.02707 (cit. on p. 18).

Ouyang, L., J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems*. Vol. 35. URL: https://arxiv.org/abs/2203.02155 (cit. on pp. 34, 71).

Penedo, G., H. Kydlíček, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. von Werra, and T. Wolf (2024). "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale". In: *arXiv preprint arXiv:2406.17557*. URL: https://arxiv.org/abs/2406.17557 (cit. on p. 11).

Peng, B., J. Quesnelle, H. Fan, and E. Shippole (2023). "YaRN: Efficient Context Window Extension of Large Language Models". In: *arXiv preprint arXiv:2309.00071*. URL: https://arxiv.org/abs/2309.00071 (cit. on p. 7).

Press, O., N. A. Smith, and M. Lewis (2022). "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/2108.12409 (cit. on p. 7).

Rafailov, R., A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn (2023). "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems*. Vol. 36. URL: https://arxiv.org/abs/2305.18290 (cit. on pp. 34, 71).

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347*. URL: https://arxiv.org/abs/1707.06347 (cit. on p. 34).

Sennrich, R., B. Haddow, and A. Birch (2016). "Neural Machine Translation of Rare Words with Subword Units". In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*. URL: https://arxiv.org/abs/1508.07909 (cit. on p. 15).

Shah, J., G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao (2024). "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision". In: *arXiv preprint arXiv:2407.08608*. URL: https://arxiv.org/abs/2407.08608 (cit. on p. 8).

Shazeer, N. (2020). "GLU Variants Improve Transformer". In: *arXiv preprint arXiv:2002.05202*. URL: https://arxiv.org/abs/2002.05202 (cit. on p. 7).

Shazeer, N., A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *International Conference on Learning Representations*. URL: https://arxiv.org/abs/1701.06538 (cit. on pp. 7, 71).

Shumailov, I., Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal (2024). "AI Models Collapse When Trained on Recursively Generated Data". In: *Nature* 631, pp. 755–759. URL: https://arxiv.org/abs/2305.17493 (cit. on p. 19).

Singhal, K., S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl, et al. (2023). "Large Language Models Encode Clinical Knowledge". In: *Nature* 620, pp. 172–180. URL: https://arxiv.org/abs/2212.13138 (cit. on p. 57).

Snell, C., J. Lee, K. Xu, and A. Kumar (2024). "Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters". In: *arXiv preprint arXiv:2408.03314*. URL: https://arxiv.org/abs/2408.03314 (cit. on pp. 38, 72).

Su, J., M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu (2024). "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *Neurocomputing* 568, p. 127063. URL: https://arxiv.org/abs/2104.09864 (cit. on p. 6).

Taori, R., I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto (2023). *Stanford Alpaca: An Instruction-following LLaMA Model*. URL: https://github.com/tatsu-lab/stanford_alpaca (cit. on p. 18).

Touvron, H., L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al. (2023). "Llama 2: Open Foundation and Fine-Tuned Chat Models". In: *arXiv preprint arXiv:2307.09288*. URL: https://arxiv.org/abs/2307.09288 (cit. on pp. 31, 72).

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. URL: https://arxiv.org/abs/1706.03762 (cit. on p. 71).

Wang, X., G. Jiang, H. Kiyomaru, Z. Tao, T. Yu, W. Ding, S. Wu, B. Yang, et al. (2024). "Scaling Synthetic Data Creation with 1 Billion Personas". In: *arXiv preprint arXiv:2406.20094*. URL: https://arxiv.org/abs/2406.20094 (cit. on p. 19).

Wang, Y., Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi (2022). "Self-Instruct: Aligning Language Models with Self-Generated Instructions". In: *arXiv preprint arXiv:2212.10560*. URL: https://arxiv.org/abs/2212.10560 (cit. on p. 18).

Wei, J., X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 35. URL: https://arxiv.org/abs/2201.11903 (cit. on pp. 38, 71).

Wu, S., O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann (2023). "BloombergGPT: A Large Language Model for Finance". In: *arXiv preprint arXiv:2303.17564*. URL: https://arxiv.org/abs/2303.17564 (cit. on p. 57).

Xu, C., Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, and D. Jiang (2023). "WizardLM: Empowering Large Language Models to Follow Complex Instructions". In: *arXiv preprint arXiv:2304.12244*. URL: https://arxiv.org/abs/2304.12244 (cit. on p. 18).

Xu, Z., F. Jiang, L. Niu, Y. Deng, R. Poovendran, Y. Choi, and B. Y. Lin (2024). "Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned LLMs with Nothing". In: *arXiv preprint arXiv:2406.08464*. URL: https://arxiv.org/abs/2406.08464 (cit. on p. 18).

Yadav, P., D. Tam, L. Choshen, C. Raffel, and M. Bansal (2023). "TIES-Merging: Resolving Interference When Merging Models". In: *Advances in Neural Information Processing Systems*. Vol. 36. URL: https://arxiv.org/abs/2306.01708 (cit. on p. 59).

Yu, G.-I., J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun (2022). "Orca: A Distributed Serving System for Transformer-Based Generative Models". In: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. URL: https://www.usenix.org/conference/osdi22/presentation/yu (cit. on p. 52).

Yu, L., B. Yu, H. Yu, F. Huang, and Y. Li (2023). "Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch". In: *arXiv preprint arXiv:2311.03099*. URL: https://arxiv.org/abs/2311.03099 (cit. on p. 59).

Yu, Q., Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, T. Fan, G. Liu, L. Liu, X. Liu, et al. (2025). "DAPO: An Open-Source LLM Reinforcement Learning System at Scale". In: *arXiv preprint arXiv:2503.14476*. URL: https://arxiv.org/abs/2503.14476 (cit. on p. 35).

Zelikman, E., Y. Wu, M. Laskin, E. Snover, N. D. Goodman, S. Polu, and J. Schulman (2024). "Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking". In: *arXiv preprint arXiv:2403.09629* (cit. on p. 39).

Ziegler, D. M., N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving (2019). "Fine-Tuning Language Models from Human Preferences". In: URL: https://arxiv.org/abs/1909.08593 (cit. on p. 34).